

SDFFlow 형상 생성 모델

쉽게 풀어 쓴 설명서: 구조, config 작성법, ex3 너트 실험, DeepJEB 진행 상황

AI-CAE4ALL

2026년 9월 28일 기준 (대표점 수 연구 추가)

이 문서는 **methods/SDFFlow**가 무엇을 하는지, config를 어떻게 써야 하는지, 지금까지 실험에서 무엇을 알았는지를 한곳에 정리한 것이다. 어려운 전산 용어 대신 풀어 쓴 말을 쓰고, config에 적는 이름(key)만 이런_글꼴로 적는다. 풀어 쓴 말과 흔히 쓰는 용어의 대조표는 부록 A에 있다. 아직 끝나지 않은 실험의 결과 자리는 **[결과 대기: 설명]**처럼 빨간 글씨로 표시했다.

한눈에 보기: 질문별 짧은 답

“VAE 입력이 32토큰? 32채널이 뭐야?”

입력은 토큰이 아니라 형상 표면에서 뽑은 점 6144개(좌표와 면 방향)이다. 모델은 이 가운데 서로 멀리 떨어진 점 T 개를 **대표점**으로 고르고, 각 대표점이 표면 점 전체를 훑어 자기 주변 모양을 숫자 32개로 요약한다. 그래서 형상 하나는 “대표점 T 개 $\times$ 숫자 32개”의 표(**형상 코드**)가 된다. ex3는 $T = 32$ 라 $32 \times 32 = 1024$ 개, DeepJEB 실험 J0-J2는 $T = 512$ 라 $512 \times 32 = 16384$ 개 숫자다 (§ 3, 그림 4).

“출력은 임의의 점에서 SDF 값이지?”

그렇다. 복원부(형상 코드를 받아 형상으로 되돌리는 부분)는 공간의 아무 점이나 받아서 그 점의 부호 거리(안쪽은 음수, 바깥은 양수, 표면은 0)를 하나 돌려준다. 학습할 때는 ± 0.1 보다 먼 값은 잘라낸다. STL을 만들 때는 128^3 격자 점마다 이 값을 계산하고 값이 0인 면을 뽑는다 (§ 2, § 5).

“그럼 $(T, 256) \rightarrow$ 요약부 $\rightarrow$ 형상 코드 $\rightarrow$ 복원부 $\rightarrow (T, 1)$ 인가?”

아니다. 두 군데가 다르다. 첫째, 요약부의 입력은 $(T, 256)$ 이 아니라 표면 점 $(6144, 6)$ 이다. $(T, 256)$ 은 요약부 안에서 대표점 T 개의 줄만 골라 온 중간 값이다. 둘째, 복원부의 출력은 $(T, 1)$ 이 아니라 $(M, 1)$ 이다. M 은 부호 거리를 물어본 질문 점의 개수로, 학습할 때 8192개, STL을 만들 때 128^3 개다. 대표점 T 개는 답을 내는 자리가 아니라, 질문 점마다 훑어보는 참고 줄이다 (§ 3.4, 그림 5).

“대표점 T 와 표면 점 수는 무조건 클수록 좋은 것 아닌가?”

복원만 보면 대표점이 많을수록 좋다. DeepJEB에서 $T = 512$ (J1)의 점검용 복원 오차는 $T = 32$ (J3)의 약 2.6분의 1이다. 그러나 형상 코드의 숫자가 16배(1024개 $\rightarrow$ 16384개)로 늘어서, 같은 벌점 가중치에서 벌점이 16배 세지고(J0는 이 때문에 무너졌다), 2단계가 만들어야 할 숫자도 16배가 된다. 실제로 2 단계까지 같은 500바퀴로 학습하니 **생성은 오히려 나빠졌다**. 조건 없이 만든 형상 중 한 조각인 비율이 72%(J3)에서 31%(J1)로, 실제 조건을 주었을 때의 부피 오차가 2.5%에서 8.3%로 나빠졌다 (§ 9.6). 2 단계를 네 배 오래 학습해도 거의 그대로였다(38%). 형상마다 코드를 여러 개 뽑아 2단계 자료를 늘리고, 학습 시점을 잡음 쪽으로 치우치고, 2단계를 키우면 512도 32 수준까지 오르지만 넘지는 못했고 2단계 시간이 약 20배다. MCB와 DrivAerML에서도 같았다. 대표점 128에 같은 처방을 하면 복원은 1.7배 좋고 생성은 32와 같아 균형이 가장 좋았다 (§ 10). 학습 형상이 수천 개 이하인 지금은 32가 작지 않다. 표면 점 수는 저장된 8192개가 상한이고, 6144개를 매번 새로 뽑는 것 자체에도 쓸모가 있다 (§ 3.4).

“Flow matching은 임의의 잡음에서 조건에 맞게 VAE 코드를 예측하는 것?”

거의 맞다. 정확히는 한 번에 “예측”하지 않고 **50걸음에 걸쳐 조금씩 옮긴다**. 생성 모델은 “지금 위치, 진행 정도, 조건”을 받아 “어느 쪽으로 가야 하는지”를 알려 주고, 잡음에서 출발해 그 방향으로 50번 걸으면 형상 코드에 도착한다. 걸음은 매번 똑같이 계산되므로 같은 잡음과 같은 조건이면 항상 같은 형상이 나온다 (§ 4).

“결과는 어쨌든 kl 1e-8부터 좋은 거네?”

ex3에서는 1e-6부터 이미 좋다. 기준은 가중치 숫자 자체가 아니라 “가중치 × 형상 코드의 숫자 개수”이며, 이 값이 약 10^{-3} 이하이면 정상이었다. 같은 1e-6도 숫자가 1024개인 ex3에서는 정상이고, 16384개인 DeepJEB J0에서는 코드가 무너졌다(§ 3.7).

“kl 1e-6이어도 castle 점검용 결과가 안 좋다. 압축 모델 자체가 부족한 것 같다”

맞다. 다만 부족한 곳이 정해져 있다. 점검용과 학습용의 오차 중앙값은 같지만, 점검용/학습용 오차 비율이 육각 너트는 1.05배, 잠금 너트는 2.4배, 캐슬 너트는 10배다. 캐슬 너트는 정답 형상을 같은 격자로 뽑아도 한 조각으로 나오는 비율이 66.7%뿐이다(§ 8.8).

“1단계, 2단계 학습 바퀴 수가 충분한가?”

1단계(압축 모델)는 충분하다. 약 500바퀴부터 점검용 오차가 평탄하고, 가장 좋은 지점은 625–885바퀴였다. 2단계(생성 모델)는 500바퀴면 조금 모자라다. 약 700바퀴로 늘리고 가장 좋았던 시점의 파일(fm_best.pth)을 쓰는 것을 권한다(§ 6.4).

“보간(interpolation) STL 상태가 그래도 안 좋다”

맞다. 가중치를 낮추면 떨어진 조각은 거의 사라진다. 남은 문제는 경로다. 섞는 도중에 구멍이 닫혔다가 끝에서 갑자기 다시 뚫리고, 세워진 너트가 한순간에 눕혀진 너트로 바뀐다. 이 “튐”의 상당 부분은 ex3 데이터에 같은 너트가 세워지거나 눕혀진 채로 섞여 있기 때문이었다(§ 8.6).

“조건부로 해 봐야 하나?”

도움은 되지만 핵심은 아니다. 실제 두 형상 사이를 조건까지 함께 옮기면 경로가 더 고르게 변한다. 하지만 구멍이 닫혔다 열리는 현상과 캐슬 너트 조각은 그대로다. 조건을 하나로 고정하면 튐이 없어지는데, 그것은 형상이 거의 움직이지 않기 때문이다(§ 8.7).

“DeepJEB에 대해서 돌릴 거지?”

돌렸다. aarl GPU 0–2에서 J1, J2, J3 세 실험이 두 단계를 모두 마쳤고(9월 26일 J3 06:36, J1 15:53, J2 16:14), 생성·섞기 평가도 J0를 포함한 네 모델에 똑같이 돌렸다(16:37 끝). 세 실험 모두 1단계가 무너지지 않았고, 생성과 섞기는 대표점 32개인 J3가 가장 좋았다(§ 9, § 9.6). 같은 부모의 형제끼리와 다른 부모끼리를 나눈 보간 평가는 아직 하지 않았다. 대표점 수는 9월 26–28일에 따로 다시 확인했다(§ 10).

차례

한눈에 보기	1
1 큰 그림: 두 단계로 나뉜 모델	4
2 부호 거리란 무엇인가	4
3 1단계: 형상 압축 모델	5
3.1 요약부: 표면 점 6144개 → 형상 코드	5
3.2 복원부: 아무 점에서나 부호 거리	5
3.3 “32토콘 × 32채널”을 그림으로	5
3.4 크기 따라가기: 어디서 무엇이 몇 개인가	7
3.5 무엇을 맞추도록 학습하나	9
3.6 학습 순서: 먼저 복원만, 그다음 흔들림과 벌점	10
3.7 퍼짐 벌점과 코드 붕괴	10
4 2단계: 생성 모델	12
4.1 잡음에서 형상 코드까지 50걸음	12
4.2 무엇을 배우나	13
4.3 조건, 빈 조건, 조건 강조 배울	13

5	형상 코드에서 STL까지	14
6	config 작성법	14
6.1	기본 문법과 실행	14
6.2	모드와 꼭 적어야 하는 key	14
6.3	학습 config: key별 의미와 정하는 법	15
6.4	바퀴 수는 충분한가	17
6.5	생성(sample) config	18
6.6	보간(interpolate) config	18
6.7	복원(reconstruct)과 평가(evaluate) config	19
6.8	자주 하는 실수 점검표	20
7	실험 방법	20
7.1	데이터	20
7.2	놓인 방향	20
7.3	비교한 모델	22
7.4	섞기 경로 7가지	22
7.5	지표	22
8	ex3 결과	23
8.1	경로 하나를 자세히 보기	23
8.2	여러 경로의 통계	23
8.3	섞는 방법 비교	26
8.4	다른 종류 사이 섞기	27
8.5	끝점 복원 확인	27
8.6	팀의 상당 부분은 방향 전환이다	27
8.7	조건부로 하면 나아지나	29
8.8	캐슬 너트와 1단계 점검용 오차	31
8.9	조건 없는 생성 결과	32
9	DeepJEB	32
9.1	왜 DeepJEB로 먼저 하나	32
9.2	J0: 기존 Studio 실험은 붕괴했다	32
9.3	J1-J3 1단계 결과	33
9.4	계획과 진행	33
9.5	결과	34
9.6	생성·섞기 평가 (J3, J1, J2, J0)	34
10	대표점 수 다시 보기: 32는 너무 작은가, 512는 안 되는가	38
10.1	DeepJEB 결과	40
10.2	다른 데이터에서 확인	41
10.3	결론: 대표점 수	42
11	권장 사항 (아직 적용하지 않음)	42
12	한계	43
A	풀어 쓴 말과 흔히 쓰는 용어	44
B	ex3 학습 config 원문	44

1 큰 그림: 두 단계로 나뉜 모델

SDFFlow는 두 개의 모델을 차례로 학습한다(그림 1).

1. **1단계, 형상 압축 모델**. 형상을 작은 숫자 표(형상 코드)로 줄였다가 다시 형상으로 되돌리는 법을 배운다. 줄이는 쪽을 **요약부**, 되돌리는 쪽을 **복원부**라 부른다. config에서 **vae_**로 시작하는 key가 이 단계다.
2. **2단계, 생성 모델**. 1단계가 만든 형상 코드들을 보고, 난수(잡음)에서 출발해 “그럴듯한 형상 코드”로 옮겨 가는 법을 배운다. 원하는 부피, 면적 같은 조건을 줄 수 있다. config에서 **fm_**으로 시작하는 key가 이 단계다.

새 형상을 만들 때는 요약부를 쓰지 않는다. 잡음 → 생성 모델 → 형상 코드 → 복원부 → 격자 위 부호 거리 → STL 순서로 간다(그림의 빨간 점선).

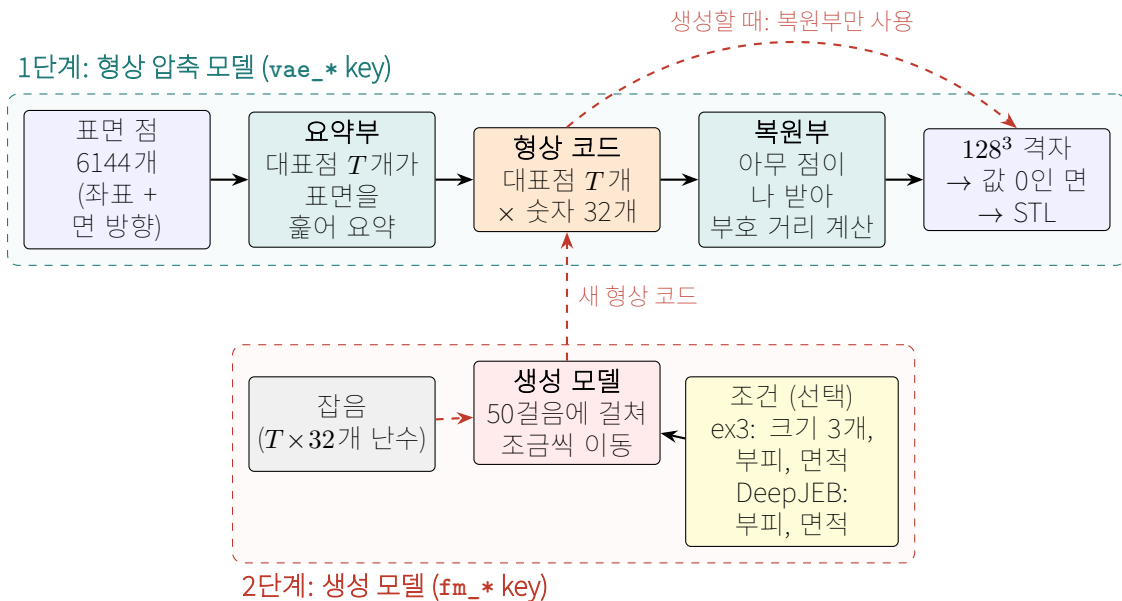


그림 1: SDFFlow의 전체 구조. 1단계는 형상 ↔ 형상 코드를 오가는 법을, 2단계는 잡음 → 형상 코드로 옮기는 법을 배운다. 빨간 점선이 새 형상을 만들 때의 경로다. 1단계를 먼저 끝까지 학습하고, 그 요약부가 만든 코드로 2단계를 학습한다(**mode train**이면 두 단계를 차례로 자동 실행).

2 부호 거리란 무엇인가

SDFFlow는 형상을 삼각형 면으로 직접 다루지 않고 **부호 거리**로 다룬다. 공간의 어떤 점이든 “가장 가까운 표면까지의 거리”를 재고, 그 점이 재료 안쪽이면 음수, 바깥이면 양수를 붙인다. 표면 위는 정확히 0이다. 너트의 구멍 안은 재료가 없으므로 바깥, 즉 양수다(그림 2a).

표면에서 먼 곳의 값은 형상을 만드는데 거의 쓸모가 없다. 그래서 학습할 때는 ± 0.1 을 넘는 값을 ± 0.1 로 잘라낸다(**clamp_dist 0.1**, 그림 2b, c). 형상은 한 변이 2인 상자($-1 \sim 1$) 안에 맞춰져 있으므로 0.1은 상자 폭의 5%다. 표면 가까운 곳만 정확히 맞추면 되고, 먼 곳은 “충분히 안쪽/바깥쪽”이라는 것만 알면 된다.

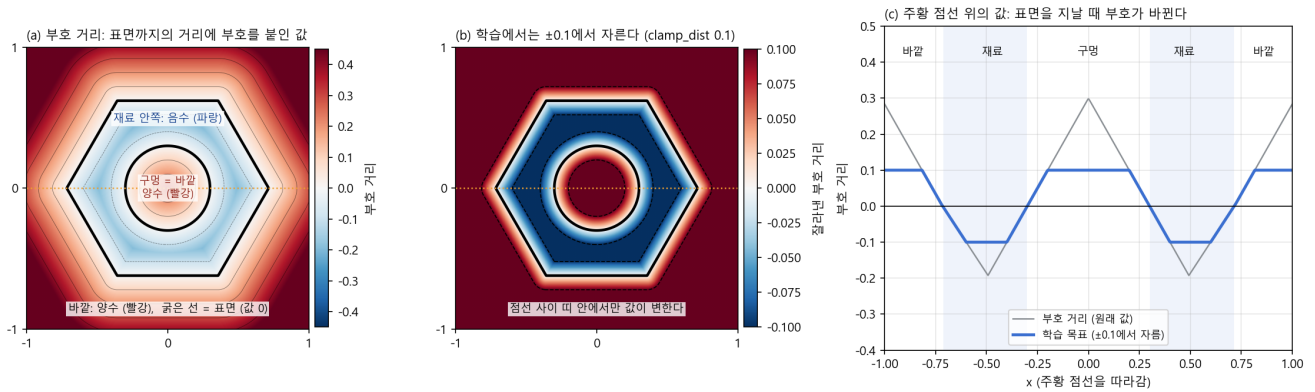


그림 2: 2차원 육각 너트 단면으로 본 부호 거리. (a) 원래 부호 거리: 재료 안쪽은 음수(파랑), 바깥과 구멍은 양수(빨강), 굵은 선이 표면(값 0). (b) ± 0.1 에서 잘라낸 값(clamp_dist 0.1): 모델이 실제로 배우는 목표. (c) (a)의 노란 점선을 따라 낸 값. 바깥 → 재료 → 구멍 → 재료 → 바깥을 지나며 원래 값과 잘라낸 값을 비교했다.

3 1단계: 형상 압축 모델

3.1 요약부: 표면 점 6144개 → 형상 코드

요약부가 하는 일을 순서대로 적으면 다음과 같다(그림 3, 숫자는 모두 실제 config 값).

1. 형상 표면에서 점 6144개와 각 점의 면 방향(바깥을 향하는 단위 방향)을 뽑는다(num_encoder_points 6144).
2. 좌표 3개를 여러 주기의 사인·코사인 값으로 늘린다. 8단계 주기(fourier_bands 8)를 쓰면 원래 좌표 3개 + 사인 24개 + 코사인 24개 = 51개가 된다. 작은 굴곡까지 구별하기 쉽게 하려는 것이다. 여기에 면 방향 3개를 붙여 54개, 이것을 다시 256개 숫자로 바꾼다(encoder_dim 256).
3. 표면 점 가운데 서로 가장 멀리 떨어지도록 T 개를 골라 대표점의 출발 위치로 삼는다(encoder_query_type fps, latent_tokens = T). 대표점이 형상 위에 고르게 퍼진다.
4. 다음 두 동작을 4번 반복한다(encoder_blocks 4). (가) 대표점 하나하나가 표면 점 6144개를 모두 훑어보고, 자기와 관계 깊은 점의 정보를 더 많이 모은다. (나) 대표점끼리 서로 정보를 나눈다(encoder_self_attention True). 한 번에 8가지 관점으로 나눠 훑는다(encoder_heads 8).
5. 마지막에 대표점마다 숫자 32개의 평균과 숫자 32개의 퍼짐 정도를 낸다(latent_dim 32).
6. 학습 중에는 평균에 퍼짐 정도만큼 흔들림을 섞은 값을 형상 코드로 쓴다. 흔들림을 섞어도 형상이 잘 복원되도록 배우게 하려는 것이다. 흔들림의 크기는 적어도 “숫자 32개의 자리마다, 그 자리의 평균값이 여러 형상과 대표점에 걸쳐 퍼진 폭”(학습하면서 조금씩 갱신한 값)의 5%로 둔다(posterior_min_std_rel 0.05). 학습이 끝난 뒤 복원이나 2단계 학습에는 흔들림 없이 평균만 쓴다.

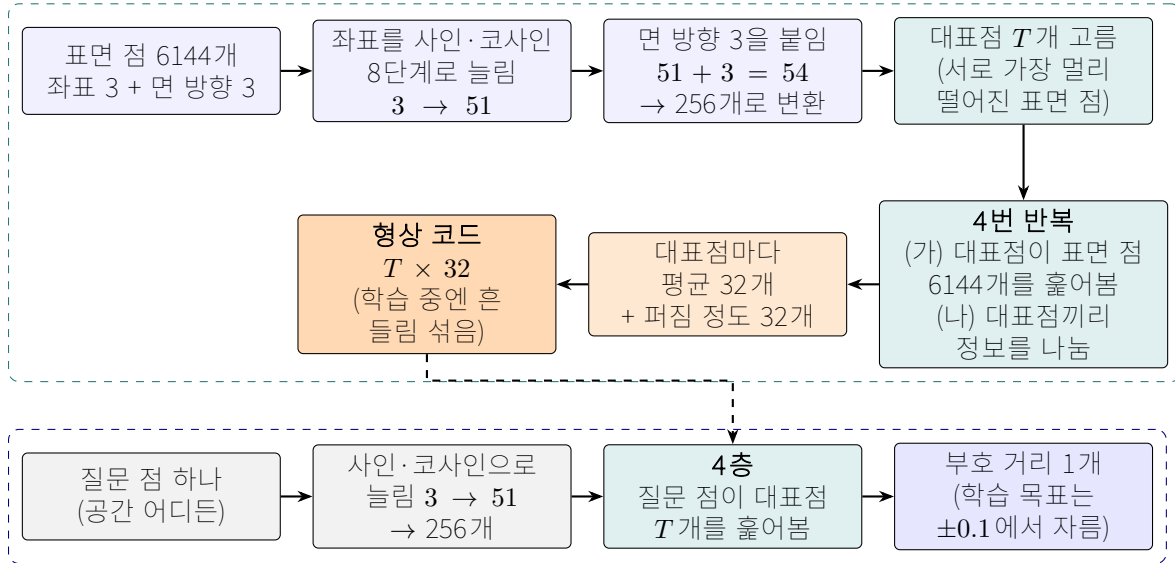
3.2 복원부: 아무 점에서나 부호 거리

복원부는 공간의 점 하나(질문 점)를 받는다. 이 점의 좌표도 사인·코사인으로 늘린 뒤, 형상 코드의 대표점 T 개를 훑어보는 층을 4번 거쳐(decoder_type attention, decoder_layers 4, decoder_hidden 256, decoder_heads 8) 부호 거리 값 하나를 낸다. 질문 점은 몇 개든 한꺼번에 물어볼 수 있다. 학습할 때는 형상마다 8192개(num_query_points 8192), STL을 만들 때는 $128^3 \approx 210$ 만 개를 묻는다.

3.3 “32토큰 × 32채널”을 그림으로

그림 4가 형상 코드의 모양이다. 흔히 “토큰”이라 부르는 것이 대표점이고, “채널”이라 부르는 것이 대표점 하나가 가진 숫자 개수다. “VAE 입력이 32토큰”이라는 말은 틀린 말이다. 입력은 표면 점 6144개이고,

요약부



복원부

그림 3: 1단계 형상 압축 모델의 실제 크기. 요약부는 표면 점 6144개를 대표점 T 개로 요약하고, 복원부는 형상 코드를 보고 아무 점의 부호 거리를 답한다. ex3는 $T = 32$, DeepJEB J0-J2는 $T = 512$.

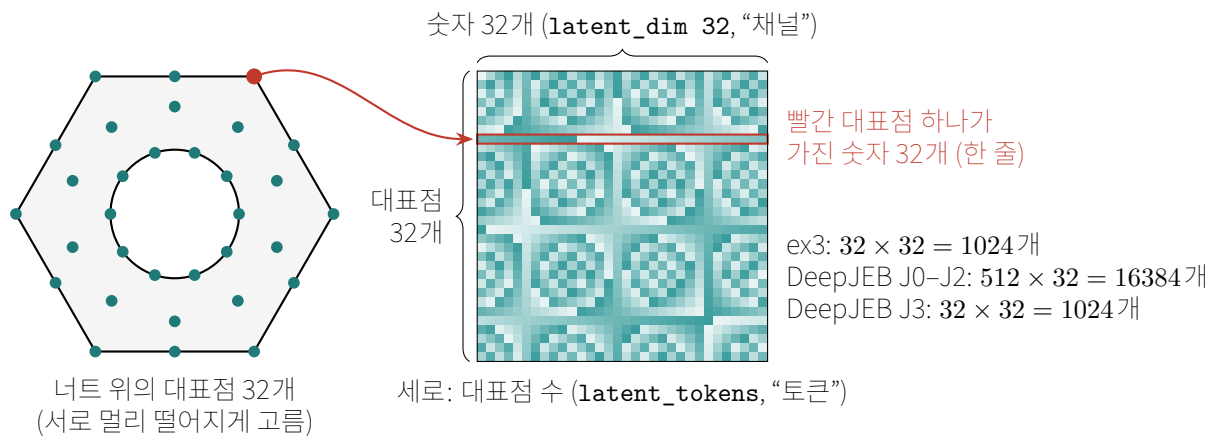


그림 4: 형상 코드의 모양. “토큰”은 형상 위에 흩어진 대표점이고, “채널”은 대표점 하나가 가진 숫자의 개수다. 형상 하나=대표점 수 × 32개의 숫자 표. 색은 숫자의 크기를 흉내 낸 예시일 뿐 실제 값이 아니다. 대표점은 요약을 시작하는 위치일 뿐이고, 4번의 훑기를 거치며 각 대표점의 숫자에는 주변뿐 아니라 형상 전체의 정보가 섞인다.

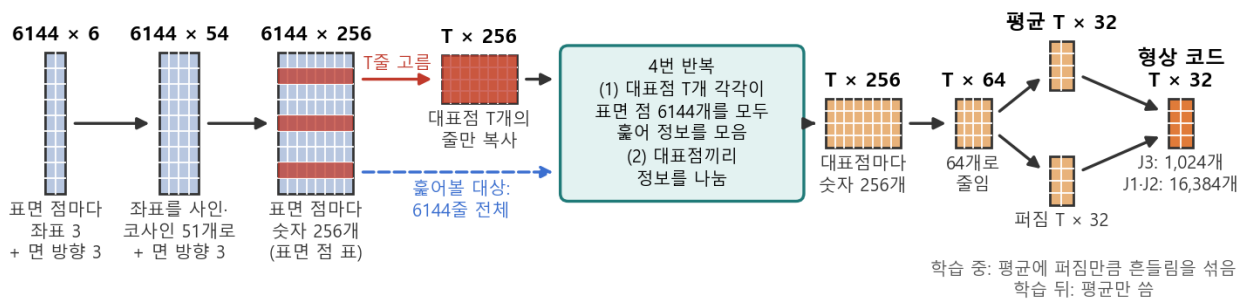
출력(형상 코드)이 대표점 32개 × 숫자 32개다.

대표점을 늘리면 형상을 더 자세히 담을 수 있지만, 2단계 생성 모델이 옮겨야 할 숫자도 그만큼 많아진다. 그리고 § 3.7에서 보듯 `kl_weight`를 같은 값으로 두면 대표점을 늘리는 순간 별점이 그만큼 세진다. 어느 쪽이 나은지는 § 3.4에서 실제 수치로 따진다.

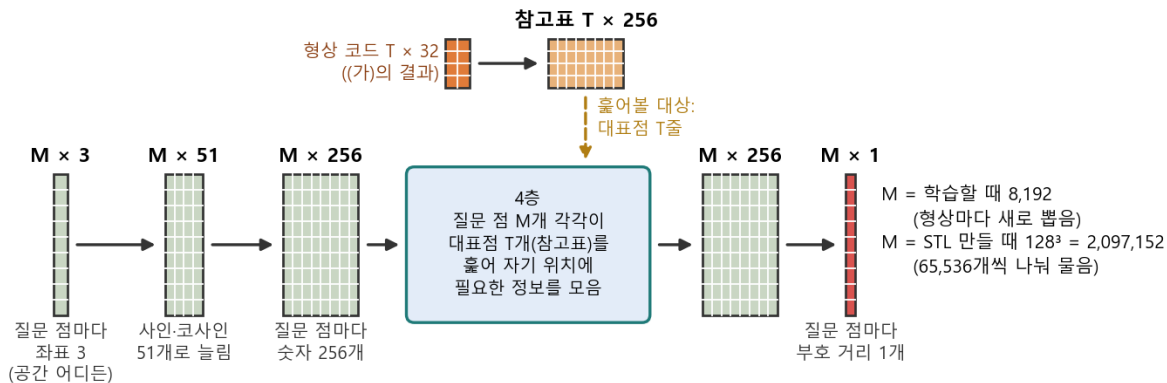
3.4 크기 따라가기: 어디서 무엇이 몇 개인가

그림 3은 단계의 순서를, 그림 5는 각 단계에 들어가고 나오는 숫자 표의 크기를 보여 준다. “ 6144×256 ”은 “줄 6144개, 줄마다 숫자 256개”라는 뜻이다. 줄 하나가 점 하나에 해당한다. 크기는 모두 DeepJEB `config(encoder_dim 256, latent_dim 32, fourier_bands 8, encoder_blocks 4, decoder_layers 4, decoder_hidden 256)`와 코드(`model/sdf_vae.py`)에서 그대로 옮겼다.

(가) 요약부: 표면 점 6144개 → 형상 코드 (대표점 T 개 × 숫자 32개)



(나) 복원부: 질문 점 M 개 + 형상 코드 → 질문 점마다 부호 거리 1개



헷갈리기 쉬운 점

✗ $(T \times 256) \rightarrow$ 요약부 $\rightarrow$ 형상 코드 $\rightarrow$ 복원부 $\rightarrow (T \times 1)$

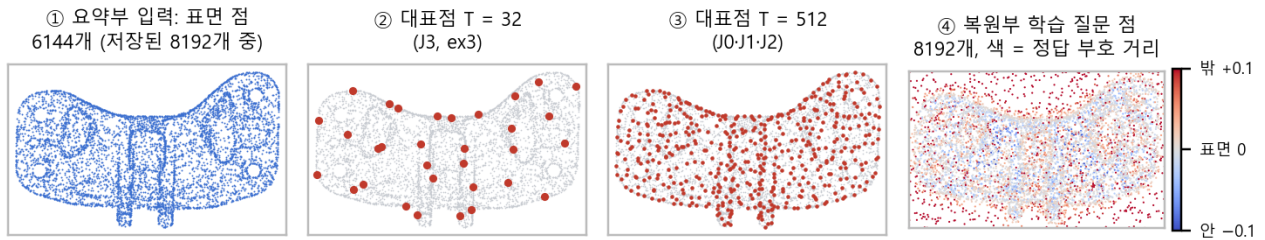
✓ 질문 점 $(M \times 3)$ + 형상 코드 $(T \times 32) \rightarrow$ 복원부 $\rightarrow$ 부호 거리 $(M \times 1)$

대표점 T 개는 답을 내는 자리가 아니라, 질문 점이 훑어보는 참고 줄이다. 답의 개수는 질문 점 수 M 이고 T 와 상관없다. T 가 32든 512든 복원부는 공간의 아무 점에서나 답하므로, 128^3 격자 전체의 부호 거리를 얻어 STL 표면을 뽑는다.

그림 5: 1단계 모델의 숫자 표 크기. (가) 요약부: 표면 점 6144개 각각을 숫자 256개로 바꾼 “표면 점 표”를 만든 뒤, 그중 서로 가장 멀리 떨어진 T 개의 줄만 복사해 대표점의 출발값으로 삼는다. 대표점 T 줄은 4번에 걸쳐 표면 점 6144줄 전체를 훑고, 마지막에 줄마다 평균 32개와 퍼짐 정도 32개를 낸다. (나) 복원부: 형상 코드 $T \times 32$ 를 $T \times 256$ 참고표로 바꾸고, 질문 점 M 개 각각이 이 참고표를 4번 훑은 뒤 부호 거리를 하나씩 낸다. 답은 질문 점 수만큼 나오며 대표점 수 T 와 관계없다.

요약부. 표면 점 6144개가 들어간다. 점마다 좌표 3개와 면 방향 3개, 합쳐 6개다. 좌표를 사인·코사인 51개로 늘리고 면 방향 3개를 붙이면 54개, 이것을 숫자 256개로 바꾸면 6144×256 표가 된다. 여기서 서로 가장 멀리 떨어진 점 T 개의 줄만 복사한 것이 $T \times 256$ 이다. 즉 $(T, 256)$ 은 요약부의 입력이 아니라 요약부 중간의 값이다. 이 T 줄이 4번에 걸쳐 표면 점 6144줄 전체를 훑어보며 정보를 모으고, 대표점끼리도 정보를 나눈다. 크기는 끝까지 $T \times 256$ 이다. 마지막에 줄마다 숫자 64개로 줄여 앞 32개를 평균, 뒤 32개를 퍼짐 정도로 쓴다. 그 결과가 형상 코드 $T \times 32$ 다.

복원부. 형상 코드 $T \times 32$ 와 질문 점 M 개가 들어간다. 형상 코드는 줄마다 숫자 256개로 바뀌어 $T \times 256$ 참고표가 된다. 질문 점도 좌표 3개를 사인·코사인 51개로 늘린 뒤 256개로 바뀌어 $M \times 256$ 이 된다. 질문 점 M 줄이 참고표 T 줄을 4번 훑고, 줄마다 숫자 하나(부호 거리)를 낸다. 그래서 출력은 $M \times 1$ 이다. M 은 몇 개든 된다. 학습할 때는 형상마다 8192개(`num_query_points` 8192), STL을 만들 때는 $128^3 = 2,097,152$ 개를 65,536개씩 나눠 묻는다. $(T, 1)$ 은 어디에도 나오지 않는다. STL의 세밀함은 T 가 아니라 질문 격자(128^3)가 정한다.



DeepJEB 형상 00000. 두 방향을 펼쳐 본 그림(가장 얇은 방향으로 눌러 봄). ④의 점은 표면 근처 8192개 + 상자 전체 2048개 중에서 뽑으며, 정답은 ± 0.1 에서 자른다.

그림 6: 실제 DeepJEB 형상 하나에서 본 점들. ① 요약부가 읽는 표면 점 6144개. ② $T = 32$ 와 ③ $T = 512$ 일 때 고른 대표점(빨강, 코드와 같은 규칙으로 계산). ④ 복원부가 학습할 때 답하는 질문 점 8192개와 정답 부호 거리. 대표점은 형상 위에만 있고, 질문 점은 형상 안팎 어디에나 있다.

표면 점 6144개는 어떻게 뽑나. 데이터 파일에는 형상마다 표면 점 8192개가 저장되어 있다 (그림 6의 ①). 학습 중에는 형상을 꺼낼 때마다 이 가운데 6144개를 새로 무작위로 뽑는다 (`num_encoder_points` 6144). 점검용 평가, 2단계 학습에 쓸 형상 코드 계산, 복원 평가에서는 형상마다 정해진 6144개를 항상 똑같이 쓴다. 그래서 같은 형상은 언제나 같은 형상 코드를 얻는다. 질문 점 8192개도 같은 방식으로, 저장된 10240개(표면 근처 8192개 + 상자 전체 2048개)에서 뽑는다.

대표점 T 는 클수록 좋은가. 복원만 보면 그렇다. 그러나 무조건은 아니다(표 1).

- **복원:** $T = 512$ 인 J1의 점검용 복원 오차는 500바퀴 끝에 1.03×10^{-3} 로, J3($T = 32$)의 2.64×10^{-3} 보다 2.6배 작다. 대표점이 많으면 복원부가 훑어볼 참고 줄이 많아 가는 부분까지 담기 쉽다.
- **벌점:** 벌점은 형상 코드의 숫자 전체에 대해 더하므로 T 에 비례한다. 같은 `kl_weight` 10^{-6} 에서 $T = 32$ 는 정상이지만 $T = 512$ 인 J0는 무너졌다(§ 3.7). T 를 늘리면 `kl_weight`를 그만큼 줄여야 한다.
- **2단계:** 생성 모델은 형상 하나에 숫자 $T \times 32$ 개를 만들어야 한다. $T = 512$ 이면 16384개로, 학습 형상 수(1713개)보다 훨씬 많다. 숫자가 많을수록 생성 모델이 배워야 할 것이 늘고 학습도 오래 걸린다.
- **시간:** 요약부의 훑기는 $T \times 6144$, 복원부의 훑기는 $M \times T$ 에 비례해 늘어난다. 실제로 1단계 한 바퀴가 J3는 약 64초, J1은 약 130초 걸렸다.

그래서 “대표점이 많을수록 좋은 생성”인지는 복원 오차만으로 판단할 수 없다. 실제로 무너지지 않은 $T = 512$ 실험(J1·J2)을 2단계까지 같은 500바퀴로 학습해 J3와 똑같이 평가하니, 복원은 2.6배 좋지만

생성은 크게 나뉘었다(표 1). 조건 없이 만든 형상에 작은 부스러기가 붙는 비율이 높고, 실제 조건을 주었을 때 요청한 부피와의 차이가 약 3배로 커졌다. 실제 형상을 그대로 복원하면 J3만큼 한 조각이 나오므로 (81~88%), 나빠진 곳은 1단계가 아니라 2단계다. 2단계 생성 모델은 1024개 대신 16384개의 숫자를 맞춰야 하는데, 학습 형상은 1713개로 같고 학습량도 같은 500바퀴(약 20분)였다. J1·J2의 2단계 점검용 오차는 끝날 때까지 내려가고 있었지만, 2000바퀴로 오래 학습해도 거의 나아지지 않았다. 2단계 처방을 합치면 32 수준까지 오르지만 넘지는 못한다(§ 10).

항목	J3 ($T = 32$)	J1 ($T = 512$)	J2 ($T = 512$)
형상 코드의 숫자 개수 ($T \times 32$)	1,024	16,384	16,384
kl_weight	10^{-8}	10^{-8}	10^{-10}
벌점 세기 (kl_weight $\times T \times 32$)	1.0×10^{-5}	1.6×10^{-4}	1.6×10^{-6}
점검용 복원 오차 (500바퀴 끝)	2.64×10^{-3}	1.03×10^{-3}	1.04×10^{-3}
1단계 한 바퀴 시간	약 64초	약 130초	약 140초
2단계 500바퀴 시간	약 13분	약 19분	약 21분
실제 형상을 그대로 복원, 한 조각	81%	81%	88%
조건 없이 생성, 한 조각	72%	31%	22%
실제 조건으로 생성, 한 조각	100%	38%	28%
부피 오차 중앙값	2.5%	8.3%	7.3%
실제 형상 쌍 거꾸로 걷기 중간, 한 조각	96%	54%	58%

표 1: 대표점 수가 다른 DeepJEB 실험. J1과 J3는 대표점 수만 다르고, J2는 J1에서 벌점 가중치만 100분의 1로 줄였다. 표면 점 수, 질문 점 수와 나머지 설정은 같다. 벌점 세기는 약 10^{-3} 이하이면 정상이었다(§ 3.7). 아래 다섯 줄은 표 16에서 옮긴 것이다.

표면 점 수는 클수록 좋은가. 늘릴 여지가 크지 않다. 첫째, 저장된 표면 점이 8192개라 그 이상은 뽑을 수 없다. 둘째, 6144개면 얇은 판 모양의 DeepJEB 형상에서 구멍 테두리까지 촘촘히 덮는다(그림 6의 ①). 셋째, 8192개 중 6144개를 매번 새로 뽑으면 같은 형상을 조금씩 다른 점들로 보게 되어, 요약부가 “어떤 점이 뽑혔는지”가 아니라 형상 자체를 요약하도록 배운다. 8192개를 모두 쓰면 매번 같은 점이 들어가 이 효과가 없어진다. 넷째, 요약부가 훑는 양은 표면 점 수에 비례해 늘어난다. 다만 표면 점 수를 바꾼 실험은 아직 하지 않았으므로, 위의 세 번째 이유는 기대하는 효과이지 측정한 효과가 아니다. 또 대표점은 표면 점 가운데서 고르므로 T 는 표면 점 수보다 클 수 없다(점검 규칙 SDF-QUERY-003).

3.5 무엇을 맞추도록 학습하나

형상마다 질문 점 8192개를 뽑아 복원부의 답과 정답 부호 거리(잘라낸 값)를 비교한다. 오차 항목은 표 2와 같다.

항목	config key와 값	뜻
부호 거리 오차	(기본, 가중치 1)	질문 점에서 답과 정답의 차이(절댓값). 가장 중요한 항목
표면 오차	surface_weight 1	표면 위 점에서는 답이 0이어야 한다
면 방향 오차	normal_weight 0.1	표면에서 부호 거리가 가장 빨리 커지는 방향이 실제 면 방향과 같아야 한다
기울기 크기 규칙	eikonal_weight 0.1	진짜 거리라면 가장 빨리 커지는 방향으로 조금 움직일 때 값도 움직인 거리만큼 변해야 한다(기울기 크기 1)
기울기 계산 점 수	hybrid_grad_points 1024	위 세 항목(표면, 면 방향, 기울기 크기)을 계산할 점의 수.
퍼짐 벌점	kl_weight	표면 점과 질문 점에서 각각 최대 이만큼 뽑는다 형상 코드가 “평균 0, 퍼짐 1”인 모양에서 멀어질수록 벌점. § 3.7 참고

표 2: 1단계의 오차 항목. 학습은 이 항목들의 가중 합을 줄이는 방향으로 진행된다.

3.6 학습 순서: 먼저 복원만, 그다음 흔들림과 벌점

처음부터 흔들림과 벌점을 모두 켜면 모델이 형상을 제대로 배우기도 전에 코드가 무너질 수 있다. 그래서 세 구간으로 나눠 학습한다(그림 7).

- 0-50바퀴: 흔들림도 벌점도 없이 그대로 복원만 배운다(`deterministic_warmup_epochs 50`).
- 50-150바퀴: 흔들림을 0에서 원래 크기(1배)까지 서서히 키운다(`posterior_noise_warmup_epochs 100, posterior_noise_max_scale 1`).
- 50-250바퀴: 퍼짐 벌점을 0에서 `kl_weight`까지 서서히 키운다(`kl_warmup_epochs 200`).
- 학습 속도(보폭)는 처음 20바퀴 동안 서서히 올린 뒤(`vae_warmup_epochs 20`) 코사인 모양으로 천천히 줄인다(`vae_learningr 0.0001`).

따라서 250바퀴 전의 점검용 오차는 서로 비교할 수 없다. 흔들림과 벌점이 계속 커지는 중이라, 같은 모델이라도 오차가 올라가는 것이 정상이다.

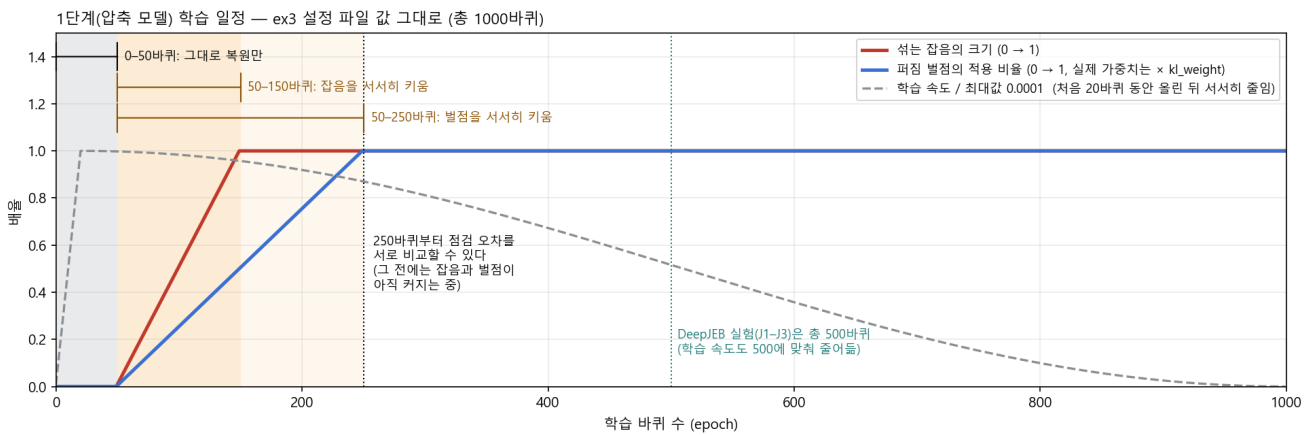


그림 7: ex3 1단계의 학습 순서(1000바퀴). 흔들림 크기, 퍼짐 벌점 가중치, 학습 속도를 바퀴 수에 따라 그렸다. 250 바퀴 이후에야 모든 설정이 고정되어 점검용 오차를 비교할 수 있다. DeepJEB 실험(J1-J3)은 총 500바퀴로 돌렸고 (DeepJEB config 파일의 `vae_training_epochs`는 1500이지만 이 실험들은 500으로 바꿨다), 학습 속도도 500 바퀴에 맞춰 줄어든다. 구간 경계(50, 150, 250)는 같다.

3.7 퍼짐 벌점과 코드 붕괴

벌점이 왜 필요한가. 2단계 생성 모델은 “평균 0, 퍼짐 1”인 정규분포 난수에서 출발한다. 형상 코드가 이것과 비슷한 모양으로 퍼져 있어야 옮기기 쉽고, 코드 사이의 빈 곳을 복원해도 이상한 형상이 덜 나온다. 퍼짐 벌점은 코드가 이 모양에서 너무 멀어지지 않게 잡아 준다.

벌점이 너무 세면: 코드 붕괴. 벌점을 줄이는 가장 쉬운 방법은 코드가 형상과 상관없이 늘 “평균 0, 퍼짐 1”이 되는 것이다. 그러면 코드에 형상 정보가 남지 않는다. 이것을 코드 붕괴라 부른다. 붕괴를 알아보려고 살아 있는 칸을 센다. 점검용 형상들에 대해, 어떤 칸의 평균값이 형상마다 달라지는 폭(분산)이 그 칸에 켜는 흔들림의 크기(분산의 평균)보다 크면 “살아 있다”고 본다. 학습 기록의 `ActiveSNR` 값이 이것이다.

기준은 가중치가 아니라 “가중치 $\times$ 숫자 개수”. 벌점은 형상 코드의 모든 숫자에 대해 *더*한 값이다. 그래서 대표점을 16배로 늘리면 같은 `kl_weight`라도 벌점이 16배 세진다. 실험에서 확인한 규칙은 다음과 같다.

$$\text{kl_weight} \times (\text{대표점 수 } T) \times 32 \leq \text{약 } 10^{-3} \Rightarrow \text{정상}$$

실험	T	kl_weight	가중치 $\times T \times 32$	결과 (살아 있는 칸)
ex3 처음 설정	32	10^{-4}	0.1	붕괴 (2 / 1024)
ex3	32	10^{-6}	1×10^{-3}	정상
ex3	32	10^{-8}	1×10^{-5}	정상
ex3	32	10^{-10}	1×10^{-7}	정상
DeepJEB 이전 실험 (config 주석)	32	10^{-6}	1×10^{-3}	정상 (1024 / 1024)
DeepJEB J0 (Studio)	512	10^{-6}	1.6×10^{-2}	붕괴 (7 / 16384)
DeepJEB J1	512	10^{-8}	1.6×10^{-4}	정상 (16384 / 16384)
DeepJEB J2	512	10^{-10}	1.6×10^{-6}	정상 (16384 / 16384)
DeepJEB J3	32	10^{-8}	1×10^{-5}	정상 (1024 / 1024)
DeepJEB J4 (제안만, 돌리지 않음)	512	6.25×10^{-8}	1.024×10^{-3}	—

표 3: 퍼짐 벌점 가중치와 붕괴 여부. 같은 10^{-6} 도 숫자가 1024개일 때는 정상, 16384개일 때는 붕괴했다. 정상이 확인된 범위는 곱이 10^{-7} – 10^{-3} 일 때다. 10^{-3} 과 1.6×10^{-2} 사이는 아직 확인하지 않았다. DeepJEB J1–J3는 500바퀴를 모두 마쳤다. 무너짐이 일어나는 50–160바퀴 구간을 무너지지 않고 지났고, 끝까지 모든 칸이 살아 있었다.

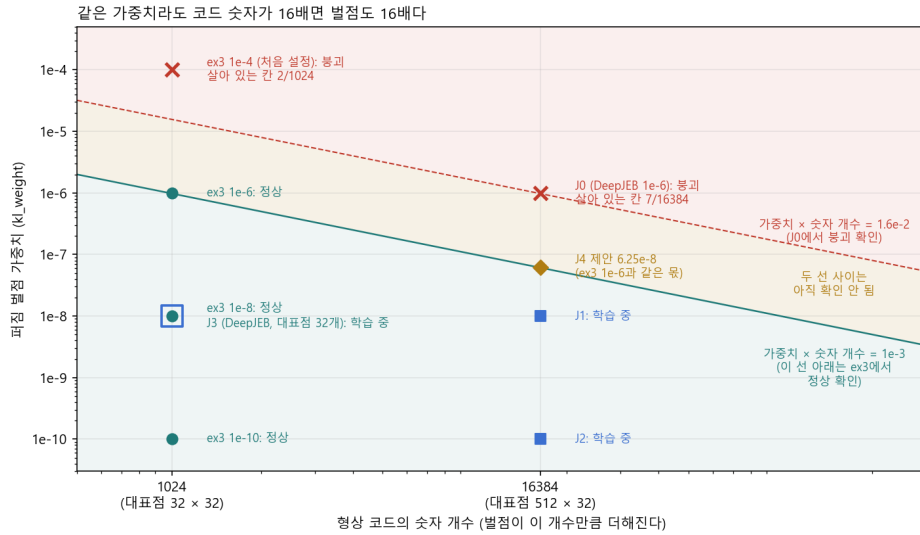


그림 8: 퍼짐 벌점 지도. 가로축은 형상 코드의 숫자 개수, 세로축은 kl_weight . 청록 실선(곱 = 10^{-3}) 아래는 ex3에서 정상이 확인된 곳, 빨간 점선(곱 = 1.6×10^{-2})은 J0에서 붕괴가 확인된 곳이다. 두 선 사이는 아직 확인하지 않았다. J4는 ex3의 10^{-6} 과 같은 곱이 되도록 계산한 제안값이다.

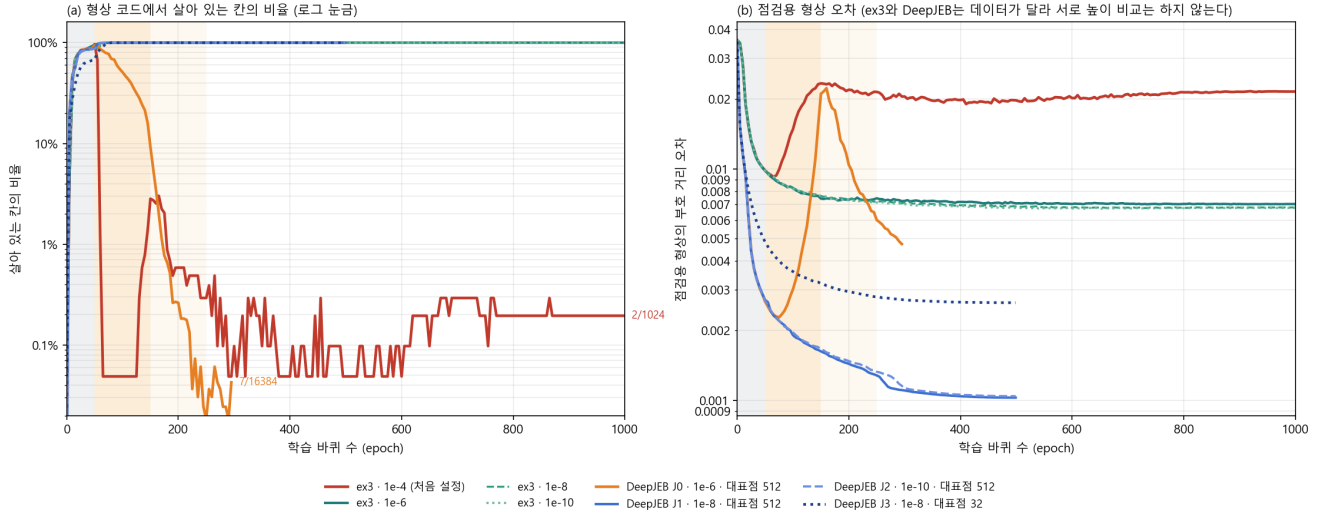


그림 9: 모든 1단계 실험의 학습 기록(DeepJEB J1-J3는 500바퀴 끝까지). (a) 살아 있는 토큰의 비율(로그 눈금). ex3 처음 설정(빨강)은 흔들림을 키우기 시작한 직후 0으로 떨어진다. 55바퀴에 694개였다가 60바퀴에 12개, 65바퀴에 0개가 되었다. DeepJEB J0(주황)는 50-150바퀴 구간 내내 줄어 150바퀴 1518개(9%), 160바퀴 564개(3%), 200바퀴 43개가 되었다. (b) 점검용 형상의 부호 거리 오차. J0는 75바퀴에 0.00228로 가장 낮았다가 160바퀴에 0.0223까지 올랐고, 295바퀴에 0.00473이었다. ex3와 DeepJEB는 데이터가 달라 두 곡선의 높이를 서로 비교하지는 않는다.

대표점 수를 바꿀 때의 환산표. ex3에서 정상이었던 10^{-6} (곱 1.024×10^{-3})과 같은 세기를 유지하려면 kl_weight 를 대표점 수에 반비례하게 줄인다.

대표점 수 T (latent_tokens)	숫자 개수 $T \times 32$	같은 세기의 kl_weight
32	1024	1×10^{-6}
64	2048	5×10^{-7}
128	4096	2.5×10^{-7}
256	8192	1.25×10^{-7}
512	16384	6.25×10^{-8}

ex3에서는 이보다 100배, 10000배 약한 벌점(10^{-8} , 10^{-10})도 정상이었고 결과도 비슷하거나 조금 나았다(§ 8). 즉 “벌점이 세면 무너지고, 약한 쪽으로는 넓은 범위가 괜찮다”. 헛갈리면 약한 쪽을 고르는 편이 안전하다.

4 2단계: 생성 모델

4.1 잡음에서 형상 코드까지 50걸음

생성 모델은 한 번에 답을 내지 않는다. 난수 표(잡음, 크기 $T \times 32$)에서 출발해 50걸음으로 나눠 조금씩 형상 코드 쪽으로 옮긴다(그림 10). 매 걸음마다 생성 모델에게 “지금 위치, 지금까지 온 정도 t (0에서 1), 조건”을 주면 “이쪽으로 가라”는 이동 방향을 돌려준다. 그 방향으로 1/50만큼 움직이고 다시 묻는다.

$$z \leftarrow z + \frac{1}{50} v(z, t, \text{조건}), \quad t = 0, \frac{1}{50}, \frac{2}{50}, \dots, \frac{49}{50}$$

여기서 z 는 지금 위치(만들어지는 중인 형상 코드)이고, v 는 생성 모델이 돌려준 이동 방향이다. $\leftarrow$ 는 “왼쪽 값을 오른쪽 계산 결과로 바꾼다”는 뜻이다. 걸음 수는 `ode_steps 50`이다. 매 걸음이 똑같이 계산되고 도중에 난수를 섞지 않으므로, 같은 잡음 + 같은 조건이면 언제나 같은 형상이 나온다. 그래서 `seed`로 잡음을 고정하면 결과를 똑같이 다시 만들 수 있다.

4.2 무엇을 배우나

학습할 때는 학습용 형상의 코드(요약부의 평균값) 하나와 잡음 하나를 곧은 선으로 잇고, 그 선 위 한 점 $z_t = (1 - t) \cdot \text{잡음} + t \cdot \text{코드}$ 를 뽑는다. 이 점에서 가야 할 방향은 “코드 - 잡음”이다. 생성 모델은 이 방향을 맞히도록 배운다. 수많은 쌍을 보면서 배우므로, 나중에는 처음 보는 잡음에서 출발해도 그럴듯한 코드로 가는 길을 안다.

생성 모델이 보는 코드는 요약부의 **평균값**(흔들림 없음)이고, 칸마다 학습용 형상들의 평균을 빼고 표준편차로 나눠 크기를 맞춘 값이다. 생성이 끝나면 이 변환을 거꾸로 해서 복원부에 넣는다.

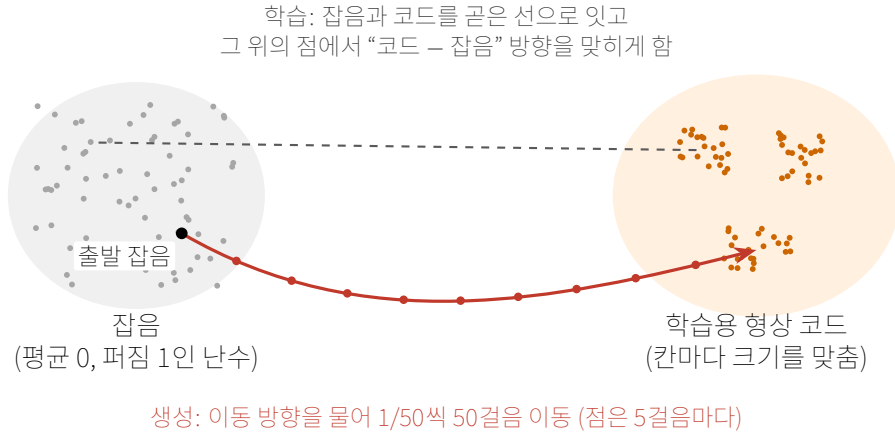


그림 10: 2단계 생성 모델이 하는 일. 학습 때는 잡음과 실제 코드를 곧은 선으로 이어 “가야 할 방향”을 가르친다(회색 점선). 생성 때는 새 잡음에서 출발해 매 걸음 방향을 물어 50걸음에 걸쳐 코드가 모인 곳으로 간다(빨간 선). 그림의 점 구름은 설명을 위한 가상 예시다.

4.3 조건, 빈 조건, 조건 강조 배율

조건. 조건은 형상에서 직접 잴 수 있는 숫자다. ex3는 상자 크기 3개(형상을 딱 맞게 감싸는 상자의 x, y, z 길이, `bbox_x`, `bbox_y`, `bbox_z`), 부피(`volume`), 표면적(`area`) 5개를, DeepJEB는 부피와 표면적 2개를 쓴다(`condition_names`). 조건값도 학습용 형상들의 평균과 표준편차로 크기를 맞춘 뒤, 평균에서 표준편차의 5배(± 5) 넘게 벗어난 값은 잘라낸다(`condition_clip 5`). 학습용 형상에서 거의 변하지 않는 조건(표준편차가 `min_condition_std 0.00001`보다 작음)이 있으면 학습이 오류로 멈추므로, 그런 조건은 `condition_names`에서 빼야 한다. ex3에는 너트 종류(육각, 잠금 등)를 나타내는 값도 데이터에 들어 있지만 조건으로는 쓰지 않는다. 평가할 때 종류별로 나눠 보는 데만 쓴다.

빈 조건. 학습 중 20%는 조건을 모두 지우고 “조건 없음” 표시를 넣는다(`cond_dropout 0.2`, `cond_dropout_mode all`). 그래서 한 모델로 조건 없는 생성도, 조건 있는 생성도 할 수 있다.

조건 강조 배율. 조건 있는 방향과 조건 없는 방향을 둘 다 계산해 차이를 키울 수 있다.

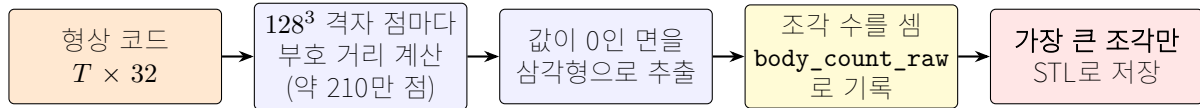
$$v = v_{\text{빈 조건}} + s(v_{\text{조건}} - v_{\text{빈 조건}}), \quad s = \text{cfg_scale}$$

$s = 1$ 이면 조건 있는 방향 그대로다. $s > 1$ 이면 조건 쪽으로 더 세게 민다. 치수는 더 정확해지지만 모양은 나빠질 수 있다. ex3에서 $s = 2$ 는 부피 오차를 줄였지만 한 조각 비율을 떨어뜨렸다 (§ 8.7). `methods/SDFFlow/CLAUDE.md`에 따르면 이전 DeepJEB 실험에서는 $s = 3$ 이 부피 오차를 2.5배 키웠다(8.5%→21.2%). 기본값 1을 권한다.

생성 모델의 크기. 같은 계산 단계 8개를 차례로 거치고(`fm_blocks 8`), 각 단계는 대표점마다 숫자 256개를 다루며(`fm_hidden 256`) 대표점끼리 8가지 관점으로 정보를 나눈다(`fm_heads 8`). 진행 정도 t 와 조건은 숫자 128개로 바뀌어(`fm_cond_hidden 128`) 각 단계의 값을 몇 배로 키우고 얼마만큼 더할지를

정하는 데 쓰인다(`fm_arch dit`). 이 조절값은 처음에 0에서 시작하므로, 학습 초반에는 각 단계가 입력을 그대로 넘긴다.

5 형상 코드에서 STL까지



주의: 마지막 단계는 코드에 고정되어 있다(`mesh_extraction.sdf_grid_to_mesh(..., keep_largest=True)`). config key가 아니므로 끌 수 없다. 떨어진 조각이 있어도 STL에는 보이지 않는다. 떨어진 조각은 실행 기록이나 결과 요약에 남는 `body_count_raw`로 확인해야 한다.

그림 11: 형상 코드에서 STL을 만드는 과정. 격자 해상도는 `mc_resolution 128`. 격자 한 칸은 상자 폭 2의 1/127, 약 0.016이다. 이보다 얇은 벽이나 틈은 격자 단계에서 이미 사라지거나 끊길 수 있다.

그림 11의 마지막 단계 때문에, 예전에 `junk/`로 가져온 보간 STL에서는 떨어진 조각이 보이지 않았다. 이 문서의 모든 조각 수는 가장 큰 조각만 남기기 전의 값(`body_count_raw`, 또는 같은 격자에서 따로 센 값)이다.

6 config 작성법

6.1 기본 문법과 실행

- 한 줄에 “key 값” 하나. 값이 여러 개면 심표로 나눈다(예: `condition_names volume, area`).
- `%` 뒤는 주석이다. 같은 key를 두 번 쓰면 오류다. 파일 맨 앞에 BOM(보이지 않는 표시 문자)이 있으면 오류다.
- 경로가 아닌 글자 값은 소문자로 바뀌어 읽힌다. 경로는 적은 그대로 읽힌다.
- 무엇을 할지(`mode`)는 명령줄이 아니라 config 안에 적는다.
- 실행은 AI-CAE4ALL 폴더 맨 위(`AI_CAE4ALL_main.py`가 있는 곳)에서 한다. 프로그램은 `methods/SDFFlow/`에서 돌기 때문에 경로는 `../../dataset/...`, `../../output/...`처럼 적는다.

```

# 검사만 (문제를 한꺼번에 모두 보여 줌)
python AI_CAE4ALL_main.py --config <config 파일> --check
# 실제로 실행될 명령만 보여 주기
python AI_CAE4ALL_main.py --config <config 파일> --dry-run
# 검사를 통과하면 바로 실행
python AI_CAE4ALL_main.py --config <config 파일>
# 필수 key 목록 보기
python AI_CAE4ALL_main.py --describe sdfflow
  
```

예를 들어 ex3 학습 config는 `configs/SDFFlow/geometry_generation/ex3/baseline/config_train_sdfflow.txt`이다.

6.2 모드와 꼭 적어야 하는 key

무엇을 할지는 `mode`로 정하고, 모드마다 꼭 적어야 하는 key가 다르다(표 4).

mode	하는 일	꼭 적어야 하는 key
train	1단계 → 2단계 차례로 학습. <code>train_vae</code> , <code>train_fm</code> 은 한 단계만	<code>dataset_dir</code> , <code>output_dir</code> , <code>vae_modelpath</code> , <code>fm_modelpath</code> , <code>latent_tokens</code> , <code>latent_dim</code> , <code>decoder_type</code> , <code>decoder_hidden</code> , <code>decoder_layers</code> , <code>encoder_dim</code> , <code>encoder_heads</code> , <code>encoder_blocks</code> , <code>num_encoder_points</code> , <code>num_query_points</code> , <code>fm_hidden</code> , <code>fm_blocks</code> , <code>fm_cond_hidden</code> , 두 단계의 바퀴 수 · 한 번에 보는 형상 수 · 학습 속도
sample	새 형상 생성	<code>vae_modelpath</code> , <code>fm_modelpath</code> , <code>output_dir</code> , <code>num_samples</code> , <code>seed</code> , <code>ode_steps</code> , <code>mc_resolution</code>
reconstruct	형상 하나를 코드로 줄였다가 복원	<code>vae_modelpath</code> , <code>input_mesh</code> , <code>output_dir</code> , <code>mc_resolution</code>
interpolate	두 생성 형상 사이 섞기, 또는 조건 옮기기	<code>vae_modelpath</code> , <code>fm_modelpath</code> , <code>output_dir</code> , <code>seed</code> , <code>source_num_samples</code> , <code>sample_index_a</code> , <code>ode_steps</code> , <code>mc_resolution</code> , 그리고 방식에 따라 <code>sample_index_b</code> 또는 <code>cond_values_a</code> · <code>cond_values_b</code>
evaluate	점검용 · 시험용 형상의 복원 성능 등을 숫자로 평가	<code>vae_modelpath</code> , <code>dataset_dir</code> , <code>output_dir</code>
optimize	생성 → 격자 생성 → 구조 해석 → 탐색을 묶은 설계 반복	<code>methods/SDFFlow/CLAUDE.md</code> 참고

표 4: SDFFlow 모드와 필수 key(`cae_suite/specs/sdfflow.py` 기준). 필수 key가 빠지면 `--check`가 알려준다.

6.3 학습 config: key별 의미와 정하는 법

표 5는 학습 config의 주요 key를 ex3와 DeepJEB(ex1) 파일의 실제 값과 함께 정리한 것이다. `model`, `mode`, `parallel_mode`, `fm_num_workers`와 학습 중 그림용 설정(`vae_num_test_shapes`, `vae_mc_resolution_test`, `fm_num_test_shapes`, `fm_mc_resolution_test`)은 표에서 뺐다. DeepJEB 값은 ex3와 다른 곳만 적었다. 파일 원문은 부록 B에 있다.

표 5: 학습 config key. “ex3”는 ex3 학습 config(원문은 부록 B), “DeepJEB”는 경로의 ex3만 ex1로 바꾼 같은 이름의 파일(현재 작업본)이다. 빈칸은 ex3와 같음.

key	ex3	DeepJEB	뜻과 정하는 법
데이터			
<code>dataset_dir</code>	<code>ex3_mcb.h5</code>	<code>ex1_deepjeb.h5</code>	학습 데이터 파일
<code>split_by_parent</code>	False	True	같은 부모 형상에서 나온 형제를 모두 같은 쪽(학습용/점검용/시험용)에 둔다. DeepJEB는 반드시 True. 아니면 점검용에 학습용 형제가 섞여 성능이 부풀려진다
<code>seed</code> , <code>split_seed</code>	42, 42		난수 고정. 데이터 나누기에도 같은 값이 쓰인다
<code>gpu_ids</code>	1	7	쓸 GPU 번호
점 수			
<code>num_encoder_points</code>	6144		요약부에 넣는 표면 점 수

key	ex3	DeepJEB	뜻과 정하는 법
num_query_points	8192		학습 때 형상마다 부호 거리를 맞춰 볼 점 수. 저장된 점(ex3 81920개, DeepJEB 10240개)에서 뽑는다
형상 코드와 요약부			
latent_tokens	32	512	대표점 수 T . 바꾸면 kl_weight 를 다시 계산해야 한다(§ 3.7)
latent_dim	32		대표점마다 숫자 개수
encoder_query_type	fps		대표점 고르는 법. fps = 서로 가장 멀리 떨어진 점
encoder_dim, encoder_heads, encoder_blocks	256, 8, 4		요약부가 점마다 다루는 숫자 개수, 관점 수, 반복 횟수
encoder_self_attention	True		대표점끼리 정보를 나눌지
fourier_bands	8		사인·코사인 늘리기 단계 수 (3 → 51)
복원부			
decoder_type	attention		질문 점이 대표점을 훑어보는 방식. 대표점이 여러 개면 이것을 쓴다
decoder_hidden, decoder_layers, decoder_heads	256, 4, 8		복원부가 점마다 다루는 숫자 개수, 층 수, 관점 수
clamp_dist	0.1		부호 거리를 자르는 거리
퍼짐 벌점과 흔들림			
kl_weight	1e-10	1e-6	퍼짐 벌점 가중치. § 3.7의 곱이 10^{-3} 이하가 되게. DeepJEB 파일의 값은 J0에서 붕괴가 확인된 조합이다(§ 11)
deterministic_warmup_epochs	50		흔들림·벌점 없이 복원만 하는 바퀴 수
posterior_noise_warmup_epochs	100		흔들림을 키우는 바퀴 수
posterior_noise_max_scale	1		흔들림의 최종 크기(1 = 퍼짐 정도 그대로)
kl_warmup_epochs	200		벌점을 키우는 바퀴 수
posterior_min_std_rel	0.05		흔들림 크기의 하한(숫자 자리별로, 모든 형상·대표점에 걸친 평균값 퍼짐 폭의 5%)
오차 항목			
surface_weight, normal_weight, eikonal_weight	1, 0.1, 0.1		표 2
hybrid_grad_points	1024		표면·면 방향·기울기 크기를 계산할 점 수
조건			
use_conditions	True		조건을 쓰지
condition_names	bbox_x, bbox_y, bbox_z, volume, area	volume, area	조건 목록. 생성할 때 조건값도 이 순서로 적는다
min_condition_std, condition_clip	0.00001, 5		표준편차가 앞의 값보다 작은 조건이 있으면 학습이 오류로 멈춤 (condition_names 에서 뺄 것). 뒤의 값은 크기를 맞춘 조건을 자르는 범위
cond_dropout, cond_dropout_mode	0.2, all		조건을 지우는 비율과 방식. all = 한꺼번에 모두 지움. per_dim = 조건마다 따로 지움(생성 때 일부 조건만 nan 으로 비워 둘 수 있음)

key	ex3	DeepJEB	뜻과 정하는 법
생성 모델			
fm_arch	dit		생성 모델 종류
fm_hidden, fm_blocks, fm_heads, fm_cond_hidden	256, 8, 8, 128		대표점마다 다루는 숫자 개수, 계산 단계 수, 관점 수, 진행 정도 t · 조건을 바꾼 숫자 개수
fm_time_sampling	uniform		학습 때 진행 정도 t 를 고르게 뽑음
ode_steps	50		생성 걸음 수
encode_batch_size	8		2단계 시작 전 학습용 형상을 코드로 바꿀 때 한 번에 처리하는 형상 수
1단계 학습			
vae_training_epochs	1000	1500	바퀴 수. 700–900이면 충분 (§ 6.4)
vae_batch_size	8		한 번에 보는 형상 수
vae_learningr, vae_weight_decay	0.0001, 0.0001		학습 속도, 모델 내부 숫자를 조금씩 작게 당기는 정도
vae_warmup_epochs	20		학습 속도를 서서히 올리는 바퀴 수
vae_use_amp	False		계산 정밀도 낮추기(속도용). 1단계는 끄
vae_use_ema, vae_ema_decay	True, 0.99		학습 중 모델 내부 숫자들의 최근 평균(가중치 평균본)을 따로 들고 있다가 그것을 저장. 값이 덜 흔들린다
vae_val_interval, vae_test_interval	5, 100		점검용 오차를 썰 간격, 그림을 그릴 간격
vae_num_workers	2		데이터 읽기 보조 작업 수
2단계 학습			
fm_training_epochs	500		바퀴 수. 약 700 권장 (§ 6.4)
fm_batch_size	64		한 번에 보는 코드 수
fm_learningr, fm_weight_decay	0.0001, 0.0001		
fm_warmup_epochs	10		
fm_use_amp	True		2단계는 켜
fm_use_ema, fm_ema_decay	True, 0.99		
fm_val_interval, fm_test_interval	5, 50		
저장 위치와 기타			
vae_modelpath, fm_modelpath	../../output/.../vae.pth, fm.pth		학습이 끝난 모델 파일
fm_best_modelpath	.../fm_best.pth		2단계 점검용 오차가 가장 낮았던 시점의 파일. 생성에는 이것을 쓴다
output_dir, pipeline_log_file, vae_log_file_dir, fm_log_file_dir	../../output/...		결과와 기록. 반드시 AI-CAE4ALL 폴더의 output/ 아래
skip_completed_stages	True		이미 끝난 단계는 다시 학습하지 않음
display_testset	True		학습 중 주기적으로 형상 그림을 저장

6.4 바퀴 수는 충분한가

- **1단계:** 1000바퀴는 충분하다. 700–900바퀴면 된다. 500바퀴 이후로는 점검용 오차가 거의 변하지 않는다. 학습용 오차(학습 중 흔들림을 섞은 채로 켜 값)는 점검용의 약 1/3에서 역시 평탄하다. 이 차이는 바퀴를 늘려서 줄어드는 것이 아니고, 대부분 오차가 유난히 큰 소수의 잠금·캐슬 너트에서 온다(점검용/학습용 오차 중앙값은 같고 평균만 3.6배, § 8.8). DeepJEB J1–J3도 500바퀴면 충분했다. 400바퀴에서 500바퀴까지 점검용 오차가 J1 1.4%, J2 1.6%, J3 0.5%만 더 내려갔다.
- **2단계:** 500바퀴는 조금 모자라다. 약 700바퀴로 늘리고 **fm_best.pth**를 쓰기를 권한다. 2000

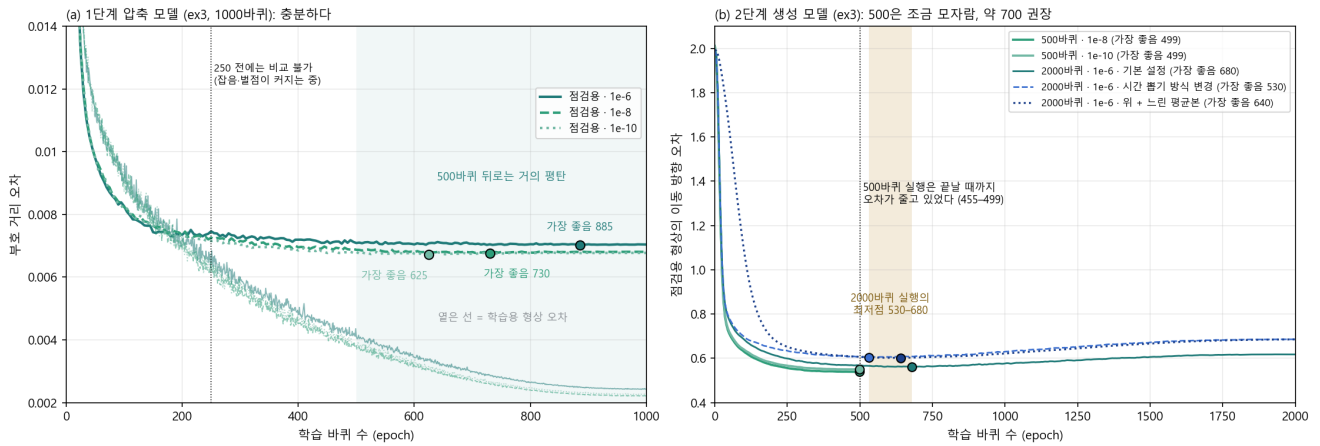


그림 12: 바퀴 수 점검(ex3). (a) 1단계: 점검용 부호 거리 오차(굵은 선)는 약 500바퀴부터 평탄하고, 가장 좋은 지점은 885(1e-6), 730(1e-8), 625(1e-10)바퀴다. 끝 무렵 학습용 오차(얇은 선)는 0.0022–0.0024, 점검용은 0.0067–0.0071이고 둘 다 평탄하다. (b) 2단계: 500바퀴 실행은 가장 좋은 지점이 455–499바퀴로, 끝날 때까지 좋아지는 중이었다. 2000바퀴로 늘린 세 실행 (기본 설정, 진행 정도 t 를 뽑는 방식 변경, 거기에 가중치 평균본을 더 천천히 갱신)은 가장 좋은 지점이 680, 530, 640바퀴였고 그 뒤로는 나아지지 않았다.

바퀴까지 늘려도 이득이 없었다.

6.5 생성(sample) config

```
model          sdfflow
mode           sample
seed          42
vae_modelpath  ../../output/.../vae.pth
fm_modelpath   ../../output/.../fm_best.pth    % 가장 좋았던 시점 파일
output_dir     ../../output/.../samples
num_samples    209
ode_steps      50
mc_resolution  128
cfg_scale      1                                % 조건 강조 배율, 기본 1
% 조건을 줄 때만 (condition_names 순서, 학습 데이터와 같은 단위):
% cond_values  0.35, 4.1
condition_ood_policy error                      % error | warn | clamp
max_condition_z 3
```

- `cond_values`를 적지 않으면 빈 조건으로 만든다. 적으면 `condition_names` 순서로 모든 값을 적는다. 일부만 `nan`(“값 없음”이라는 뜻으로 적는 글자)으로 비워 두려면 2단계를 `cond_dropout_mode per_dim`으로 학습했어야 한다.
- 요청한 조건이 학습 데이터 평균에서 표준편차의 3배(`max_condition_z 3`) 넘게 벗어나면, `condition_ood_policy`에 따라 멈추거나(`error`), 경고만 하거나(`warn`), 경계값으로 잘라서(`clamp`) 만든다.
- 위 예의 `cond_values` 숫자는 형식을 보여 주는 예시일 뿐이다.

6.6 보간(interpolate) config

`interpolation_space`로 세 가지 방식을 고른다.

slerp_noise (기본). 생성 모델의 잡음 두 줄을 원호를 따라 섞은 뒤 50걸음 생성한다. 여기서 잡음 “한 줄”은 형상 하나를 만드는 잡음이다 (seed 42라면 0번 줄이 `sample_42_000.stl`을 만든다. 두 줄은

`sample_index_a`, `sample_index_b` 번째 줄). 숫자가 많은 잡음은 전체 크기(모든 숫자를 제공해 더한 값)가 거의 일정한데, 두 잡음을 직선으로 섞으면 중간에서 이 크기가 작아져 “평범하지 않은 잡음”이 된다. 원호로 섞으면 크기가 유지된다. 두 끝은 원래 생성 형상과 똑같다.

lerp_latent (예전 방식). 두 생성 형상의 코드(칸마다 크기를 맞춘 값)를 직선으로 섞는다.

cond_sweep. 잡음 한 줄을 고정하고 조건만 `cond_values_a`에서 `cond_values_b`까지 `sweep_steps` 칸(기본 5, 2 이상)에 걸쳐 직선으로 옮긴다. 결과는 `sample_<seed>_sweep_<k>.stl`로 저장된다. 조건 있는 2단계 모델이 필요하다.

```
% (가) 두 생성 형상 사이 섞기
mode                interpolate
interpolation_space slerp_noise
seed                42
source_num_samples  209          % sample 실행의 num_samples와 같게
sample_index_a      0            % 그 실행의 sample_42_000.stl
sample_index_b      1            % 그 실행의 sample_42_001.stl
alpha               0.5          % 한 번 실행에 섞는 비율 하나
ode_steps           50
cfg_scale            1
mc_resolution       128

% (나) 조건만 옮기기 (DeepJEB: volume, area 순서)
mode                interpolate
interpolation_space cond_sweep
seed                42
source_num_samples  209
sample_index_a      0            % 고정할 잡음 줄
cond_values_a        <부피 A>, <면적 A>
cond_values_b        <부피 B>, <면적 B>
sweep_steps          9
cfg_scale            1
```

제약과 주의점은 다음과 같다.

- `slerp_noise`와 `lerp_latent`는 조건 없는 섞기만 된다. `cond_values`를 적으면 프로그램이 멈춘다. 조건을 쓰려면 `cond_sweep`을 쓴다.
- 한 번 실행에는 비율(`alpha`)을 하나만 쓴다. 두 끝과 중간 하나, 그리고 세 형상을 나란히 그린 그림이 저장된다. 비율 11개를 보려면 11번 실행한다.
- 끝점은 생성된 형상이다. 실제 데이터 형상 두 개 사이를 섞는 기능은 지금 없다. 이 문서의 실제 형상 사이 보간 (§ 7.4)은 따로 만든 평가 프로그램으로 돌렸다.
- 끝점을 `mode sample` 결과와 똑같이 맞추려면 `seed`와 `source_num_samples`를 그 실행의 `seed`, `num_samples`와 같게 둔다. 만든 잡음 개수가 다르면 앞쪽 번호의 잡음이 같게 나온 적은 있지만 보장되지는 않는다.

6.7 복원(reconstruct)과 평가(evaluate) config

```
mode                reconstruct
vae_modelpath       ../../output/.../vae.pth
input_mesh           <복원할 STL 경로>
output_dir           ../../output/.../recon
mc_resolution       128
latent_refine_steps  1000        % 0이면 끄. 복원 품질이 목적이면 켜
latent_refine_lr     0.01
```

```

latent_refine_prior_weight 0

mode                evaluate
vae_modelpath       ../../output/.../vae.pth
dataset_dir         ../../dataset/geometry_generation/ex1_deepjeb.h5
output_dir          ../../output/.../eval
split_seed          42
split_by_parent      True      % 학습 때와 반드시 같게
eval_task            reconstruction
eval_split           test      % train | val | test
eval_num_shapes      0         % 0 = 전부

```

latent_refine_steps는 요약부가 낸 코드를 출발점으로, 복원 결과가 입력 형상과 더 맞도록 코드를 직접 1000번 고치는 기능이다. ex3에서 이렇게 하면 표면 평균 거리가 “정답 형상을 같은 격자로 뽑았을 때의 표면 평균 거리”의 1.04–1.09배까지 내려갔다 (kl 1e-6 점검용 중앙값 0.0069 → 0.0064). 즉 복원부는 충분히 좋고, 부족한 것은 요약부다. 대신 고친 코드는 요약부가 내는 코드의 퍼짐 범위에서 59–61% 벗어나므로, 고친 코드를 2단계 학습에 넣으면 안 된다.

6.8 자주 하는 실수 점검표

1. **latent_tokens**를 바꾸고 **kl_weight**를 그대로 둬 → 벌점이 대표점을 늘린 배수만큼 세짐 (32 → 512면 16배). DeepJEB 파일의 512 / 1e-6이 이 경우다.
2. STL만 보고 떨어진 조각이 없다고 판단함 → **body_count_raw**를 볼 것 (§ 5).
3. DeepJEB에서 **split_by_parent False** → 형제 형상이 점검용에 섞임. **mode evaluate**도 학습 때와 같은 값으로.
4. 250바퀴 전의 점검용 오차로 실험을 비교함 → 흔들림 · 벌점이 커지는 중이라 비교가 안 됨.
5. 보간 끝점이 sample 결과와 다름 → **source_num_samples**를 **num_samples**와 같게.
6. **slerp_noise**에 **cond_values**를 적음 → 멈춤. 조건 섞기는 **cond_sweep**.
7. **cfg_scale**을 크게 둬 → 치수는 맞아도 모양이 나빠짐. 기본 1, 치수가 꼭 맞아야 할 때만 2.
8. **fm.pth**(마지막 시점)로 생성 → **fm_best.pth** 권장.
9. 조건값의 순서나 단위가 틀림 → **condition_names** 순서, 학습 데이터와 같은 단위.
10. 코드를 고친 복원 결과(**latent_refine_steps** > 0)를 2단계 학습 데이터로 씬 → 쓰지 말 것.
11. 실행 전에 **--check**를 안 돌림 → 필수 key 누락, 허용되지 않는 값, 경로 오류를 한 번에 잡아 준다.

7 실험 방법

7.1 데이터

두 데이터의 차이는 표 6에 정리했다. 실제 형상은 ex3가 그림 13, DeepJEB가 그림 14에 있다.

7.2 놓인 방향

너트의 구멍 축 방향을 표면 점 분포의 세 주축으로 구했다. 세 주축 가운데 길이가 나머지 둘과 가장 다른 축을 구멍 축으로 보고, 그 축이 x, y, z 어느 것과도 18° 넘게 어긋나면 “비스듬”으로 썼다(그림 15). ex3 1305개는 z 방향(세움) 742, x 방향(누움) 330, y 방향(누움) 128, 비스듬 105개다. 육각 너트의 28%, 홈 너트의 92%가 x 방향으로 누워 있고, 캐슬 너트는 비스듬이 75개다. DeepJEB는 2138개가 거의 모두 (축마다 2137–2138개) 가장 긴 축이 y, 중간 축이 x, 가장 짧은 축이 z로 같다.

	ex3 (MCB 너트)	DeepJEB (브래킷)
형상 수	1305 (학습용 1044 / 점검용 130 / 시험용 131)	2138, 부모 형상 263개
종류	5종: 육각 824, 잠금 196, 캐슬 181, 홈 59, 사각 45	1종 (부모 421번은 형제 15개)
split_by_parent 놓인 방향 조건	False (부모 정보 없음) 제각각 (세움 / 누움 / 비스듬, 그림 15) 상자 크기 3개, 부피, 면적 (5개)	True 모두 같음 부피, 면적 (2개, 실질적으로 약 1.5개 몫)
형상당 저장된 부호 거리 점	81920	10240 (표면 근처 8192 + 고르게 2048)

표 6: 두 데이터의 차이. DeepJEB는 부피와 면적이 서로 강하게 함께 움직여서 조건 두 개가 사실상 1.5개 정도의 정보만 준다.

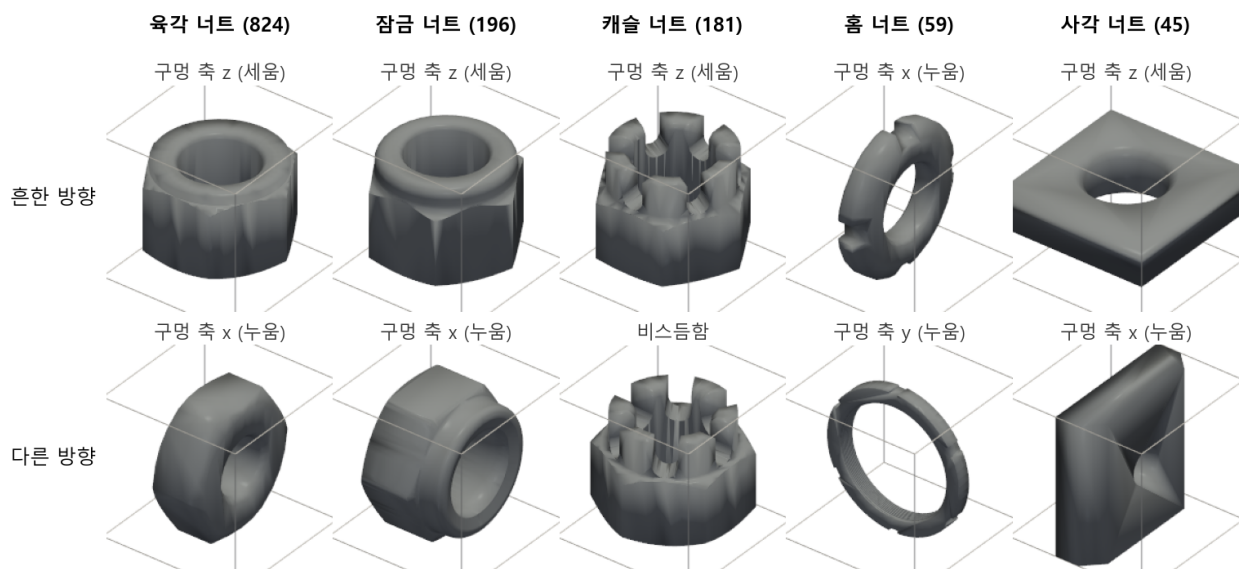


그림 13: ex3의 5종 너트. 종류마다 가장 흔한 방향과 다른 방향으로 놓인 실제 형상을 하나씩 보였다. 같은 종류라도 세워진 것과 눕혀진 것이 섞여 있다.

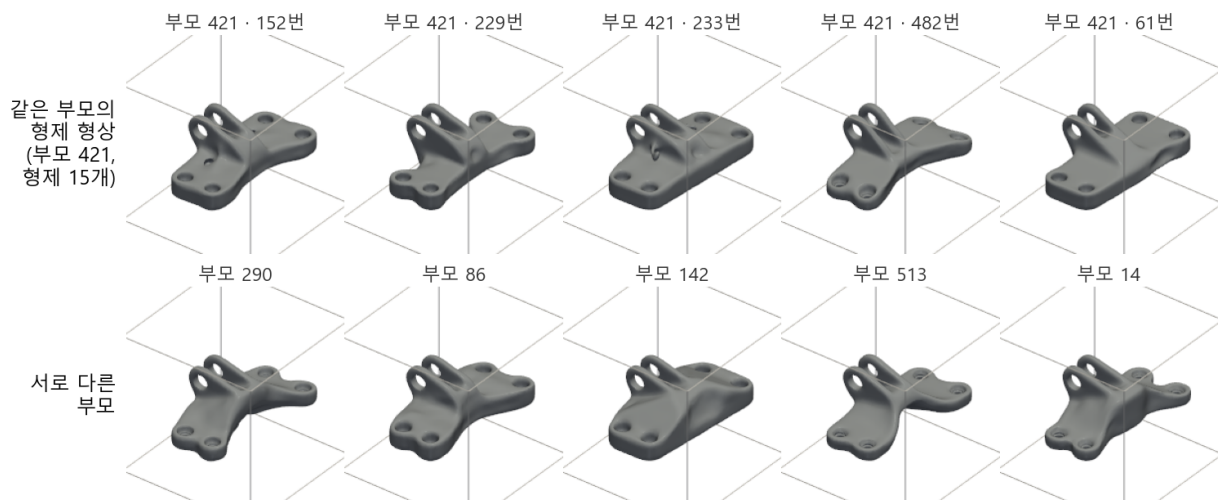


그림 14: DeepJEB 브래킷. 위 줄은 부모 421번의 형제 5개(152, 229, 233, 482, 61번), 아래 줄은 서로 다른 부모 5개(290, 86, 142, 513, 14번)에서 하나씩. 형제끼리는 같은 계열의 조금씩 다른 변형이다. 모든 형상이 같은 방향으로 놓여 있다.

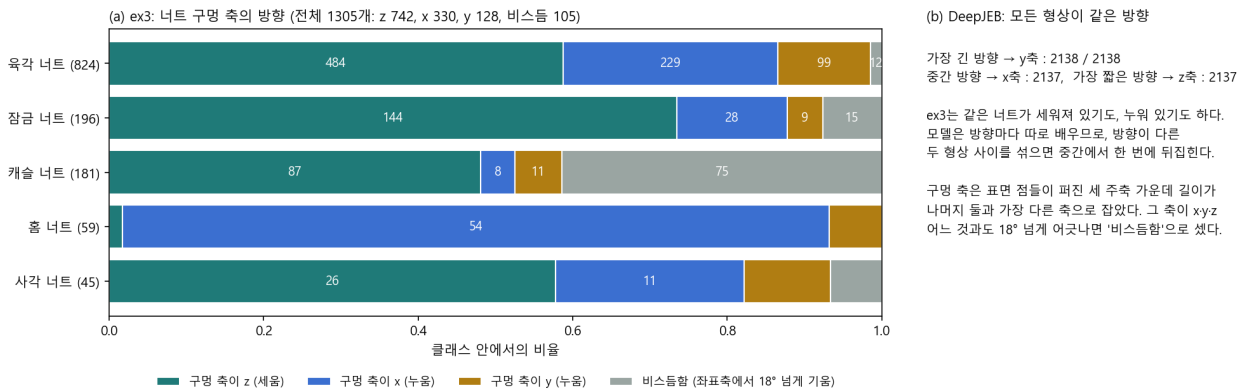


그림 15: 놓인 방향. (a) ex3 종류별 구멍 축 방향(z/x/y/비스듬): 육각 484/229/99/12, 잠금 144/28/9/15, 캐슬 87/8/11/75, 홈 1/54/4/0, 사각 26/11/5/3. (b) DeepJEB는 모든 형상의 세 주축이 같다.

7.3 비교한 모델

ex3에서 1단계의 **kl_weight**만 바꾼 네 모델을 비교했다. 처음 설정 1e-4(붕괴), 1e-6, 1e-8, 1e-10. 나머지 설정은 모두 같고, 각 모델마다 2단계를 500바퀴 학습했다. 한 모델 안에서 같은 잡음은 늘 같은 형상을 만들지만, 모델이 서로 다르므로 같은 잡음이라도 모델마다 끝점 형상이 다르다.

7.4 섞기 경로 7가지

섞는 방법은 네 종류이고(그림 16), 끝점과 조건을 달리해 경로 7묶음을 만들었다(표 7).

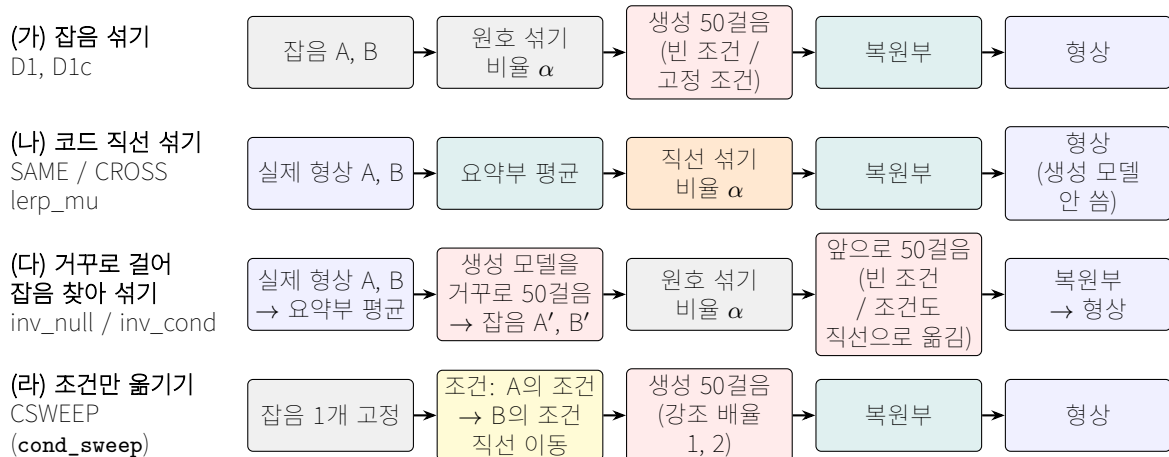


그림 16: 평가에 쓴 네 종류의 섞기. (가)는 프로그램의 **slerp_noise**와 같다. (나)와 (다)는 실제 형상 두 개 사이를 섞기 위해 따로 만든 평가 프로그램으로 돌렸다(프로그램에 없는 기능). (라)는 프로그램의 **cond_sweep**과 같다. (다)에서 찾은 잡음을 다시 앞으로 걸으면 원래 형상과의 겹침률(두 형상 내부가 겹치는 부피 ÷ 둘을 합친 부피, 1이면 똑같음) 중앙값이 1.00이라, 거꾸로 걷기 자체는 정확하다.

7.5 지표

모든 형상은 128^3 격자에서 가장 큰 조각만 남기기 전의 상태로 냈다.

겹침률. 두 형상을 같은 128^3 격자에 놓았을 때, 두 형상 내부가 겹친 부피를 둘 중 하나라도 차지한 부피로 나눈 값. 완전히 같으면 1, 전혀 겹치지 않으면 0이다.

한 조각 비율. 형상이 조각 하나로 나오는 비율. “중간”은 비율 0.1–0.9의 9칸만 센 것이다. 높을수록 좋다.

이름	끝점	방법	조건	개수
D1	새 잡음 쌍 (seed 7)	(가)	빈 조건	16경로 × 11비율
D1c	D1의 앞 8쌍과 같은 잡음	(가)	쌍마다 점검용 형상 하나의 실제 조건으로 고정	8 × 11
SAME lerp_mu	같은 종류의 실제 점검용 형상 쌍	(나)	—	8 × 11
SAME inv_null	같은 쌍	(다)	빈 조건	8 × 11
SAME inv_cond	같은 쌍	(다)	각 끝을 자기 조건으로 거꾸로 견고, 조건도 직선으로 옮김	8 × 11
CROSS lerp_mu / inv_null	다른 종류의 실제 형상 쌍	(나) / (다)	— / 빈 조건	8 × 11
CSWEEP	잡음 고정, 조건만 A→B	(라)	강조 배율 1, 2	모델·배율마다 8 경로 × 9비율

표 7: 섞기 경로 목록. 비율 11개는 0, 0.1, ..., 1이고, CSWEEP의 9개는 0, 0.125, ..., 1이다. CSWEEP 경로 8개는 SAME 앞 4쌍(육각 443→938, 잠금 210→1080, 캐슬 383→355, 홈 1254→1246) × 잡음 2개(seed 11)이다.

추가 조각. 격자 칸 100개보다 큰 조각의 수에서 1을 뺀 값의 평균. 작은 먼지 같은 조각은 세지 않는다. 낮을수록 좋다.

팀 지수. 이웃한 두 비율 사이에서 형상 내부가 얼마나 바뀌었는지($1 - \text{겹침률}$)를 재고, 그 가운데 최댓값을 중앙값으로 나눈 것. 고르게 변하면 1-2, 한 번에 뒤집히면 커진다. “팀 > 3”은 팀 지수가 3을 넘는 경로의 비율이다.

구멍 수 오르내림. 경로를 따라 가장 큰 조각의 구멍 수가 오르내린 총량이 두 끝의 구멍 수 차이보다 큰 경로의 비율. 구멍이 닫혔다가 다시 뚫리면 여기에 걸린다. 낮을수록 좋다.

조건 오차. CSWEEP에서, 각 비율에서 요구한 조건값(두 끝 조건의 직선 섞기)과 생성 형상에서 잔 값의 상대 오차. 경로별 중앙값.

부피 순위 상관. 비율 α 의 순서와 생성 형상 부피의 순서가 얼마나 같은지(1이면 완전히 같음). 요구 부피가 5% 넘게 변하는 경로로 계산.

끝점 겹침률. CSWEEP 끝 형상과 그 조건을 준 실제 형상의 겹침률.

8 ex3 결과

8.1 경로 하나를 자세히 보기

그림 17과 표 8은 프로그램의 기본 보간(**slerp_noise**) 경로 하나다. 처음 설정($1e-4$)은 비율 0.1-0.8 내내 5-18조각으로 부서지고(0.9에서도 2조각), 떨어진 조각이 표면적의 최대 64.4%를 차지한다. 예전에 가져온 STL에는 이 가운데 가장 큰 조각만 남아 있었다. $1e-6$ 의 떨어진 조각은 모두 표면적 5% 미만의 작은 점이고, $1e-8$ 과 $1e-10$ 은 거의 없다. 대신 $1e-8$ 은 비율 0.1-0.5에서 구멍이 닫힌 판(구멍 수 0)이 되었다가 0.8→0.9에서 한 번에 육각 너트로 튀고, $1e-10$ 은 0.5→0.6에서 판에서 캐슬 고리로 튜다.

8.2 여러 경로의 통계

경로 하나로는 판단할 수 없으므로, 모델마다 표 7의 경로를 모두 돌렸다(표 9).

표 9에서 읽을 수 있는 것은 다음과 같다.

1. 처음 설정($1e-4$)은 모든 경로에서 나쁘다. 한 조각 비율이 22-49%, 구멍 수 오르내림이 88-100%다. 코드 붕괴의 결과다.

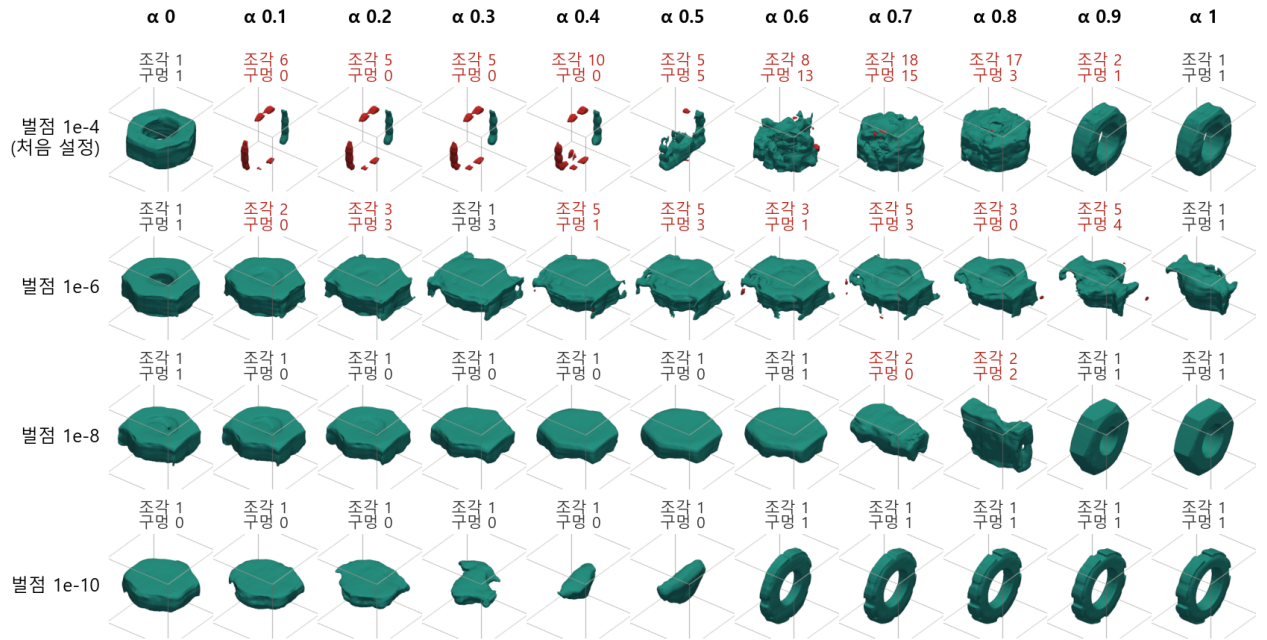


그림 17: seed 42 잡음의 0번, 1번 줄을 원호로 섞은 경로(빈 조건). 행은 모델 4개, 열은 섞는 비율 0→1. 청록은 가장 큰 조각, 빨강은 나머지 조각이다. 칸의 숫자는 프로그램이 기록한 조각 수(`body_count_raw`)와 가장 큰 조각의 구멍 수다.

모델	조각 수 (비율 0 → 1)	떨어진 조각 최대 면적	튜 지수	경로 길이 / 직선 거리
1e-4 (처음)	1 6 5 5 10 5 8 18 17 2 1	64.4%	7.6	5.81
1e-6	1 2 3 1 5 5 3 5 3 5 1	4.5%	2.7	2.21
1e-8	1 1 1 1 1 1 1 2 2 1 1	< 0.1%	12.2	2.83
1e-10	1 1 1 1 1 1 1 1 1 1 1	0	3.3	1.79

표 8: 그림 17의 경로 하나(경로 1개뿐). 여기서 튜 지수와 경로 길이는 이웃 STL 사이 표면 평균 거리로 잰다. 경로 길이 / 직선 거리가 클수록 두 끝 사이를 돌아서 간 것이다.

경로	모델	중간 한 조각 비율 ↑	추가 조각 ↓	틈 지수 중앙값 ↓	틈 > 3 ↓	구멍 수 오르내림 ↓
D1 잡음 섞기 빈 조건 (16)	1e-4	45%	0.688	6.63	81%	94%
	1e-6	83%	0.125	5.02	69%	50%
	1e-8	85%	0.083	3.14	69%	56%
	1e-10	86%	0.132	7.15	81%	50%
D1c 잡음 섞기 조건 고정 (8)	1e-4	49%	0.500	5.12	100%	88%
	1e-6	74%	0.403	2.44	25%	38%
	1e-8	74%	0.500	2.36	0%	38%
	1e-10	75%	0.278	2.33	38%	50%
D1c와 같은 잡음 빈 조건 (8)	1e-4	43%	0.403	7.02	100%	88%
	1e-6	89%	0.097	7.49	88%	38%
	1e-8	90%	0.042	3.14	75%	63%
	1e-10	89%	0.083	10.57	88%	50%
SAME 코드 직선 섞기 (8)	1e-4	38%	0.583	6.13	100%	100%
	1e-6	61%	0.556	3.60	63%	75%
	1e-8	64%	0.444	3.75	63%	75%
	1e-10	63%	0.375	2.92	38%	75%
SAME 거꾸로 걷기 빈 조건 (8)	1e-4	28%	0.708	9.12	100%	100%
	1e-6	65%	0.278	3.28	63%	75%
	1e-8	72%	0.181	4.70	63%	50%
	1e-10	61%	0.028	4.36	75%	88%
SAME 거꾸로 걷기 조건도 옮김 (8)	1e-4	22%	1.375	4.91	88%	100%
	1e-6	63%	0.153	1.96	25%	75%
	1e-8	74%	0.153	2.33	38%	63%
	1e-10	57%	0.236	2.07	25%	75%
CROSS 코드 직선 섞기 (8)	1e-4	35%	1.000	7.06	88%	100%
	1e-6	78%	0.181	4.24	75%	63%
	1e-8	79%	0.097	3.38	75%	63%
	1e-10	83%	0.083	3.72	63%	63%
CROSS 거꾸로 걷기 빈 조건 (8)	1e-4	24%	1.528	6.81	88%	100%
	1e-6	72%	0.097	4.25	75%	63%
	1e-8	69%	0.181	4.02	75%	63%
	1e-10	76%	0.111	2.25	38%	50%

표 9: ex3 모델 4개 × 경로 8묶음(표 7에서 CSWEEP을 빼고, CROSS를 두 방법으로 나누고, D1c와 같은 잡음을 빈 조건으로 돌린 비교 묶음을 더했다). 괄호 안은 경로 수. ↑는 높을수록, ↓는 낮을수록 좋다. 캐슬이 끼인 쌍(SAME 8쌍 중 2쌍, CROSS 8쌍 중 4쌍)을 빼고 정상 모델 3개(1e-6, 1e-8, 1e-10)만 보면 중간 한 조각 비율 / 추가 조각이 SAME 코드 직선 섞기 78-80% / 0.26-0.39, SAME 거꾸로 걷기·빈 조건 76-87% / 0-0.15, SAME 거꾸로 걷기·조건도 옮김 76-93% / 0.02-0.17, CROSS 코드 직선 섞기 86-94% / 0.03-0.08, CROSS 거꾸로 걷기 86-100% / 0이다.

2. $1e-6, 1e-8, 1e-10$ 은 비슷하다. 어느 하나가 모든 경로에서 앞서지 않는다. 경로 수가 8-16개라 이 정도 차이는 우연일 수 있다.
3. 어느 방법도 구멍 수를 고르게 바꾸지 못한다. 정상 모델에서도 경로의 38-88%에서 구멍이 닫혔다가 다시 열린다. 섞는 방법의 문제가 아니라, 1단계가 형상들을 형상 코드로 늘어놓은 방식 자체의 성질로 보인다.
4. 코드 직선 섞기(생성 모델 없음)는 같은 종류 쌍에서 추가 조각이 가장 많다(0.375-0.556). 두 코드의 중간이 요약부가 실제로 내는 코드 범위 밖으로 나가기 때문이다. 생성 모델을 거꾸로 걸어 섞으면 추가 조각이 크게 준다. 빈 조건이면 절반 이하(0.028-0.278)이고, 조건도 옮기면 0.153-0.236이다.
5. 실제 두 형상 사이를 조건까지 함께 옮기면 가장 고르게 변한다. 틸 지수 중앙값이 3.28-4.70에서 1.96-2.33으로 내려가고, 틸 > 3 경로가 63-75%에서 25-38%로 줄어든다.

8.3 섞는 방법 비교

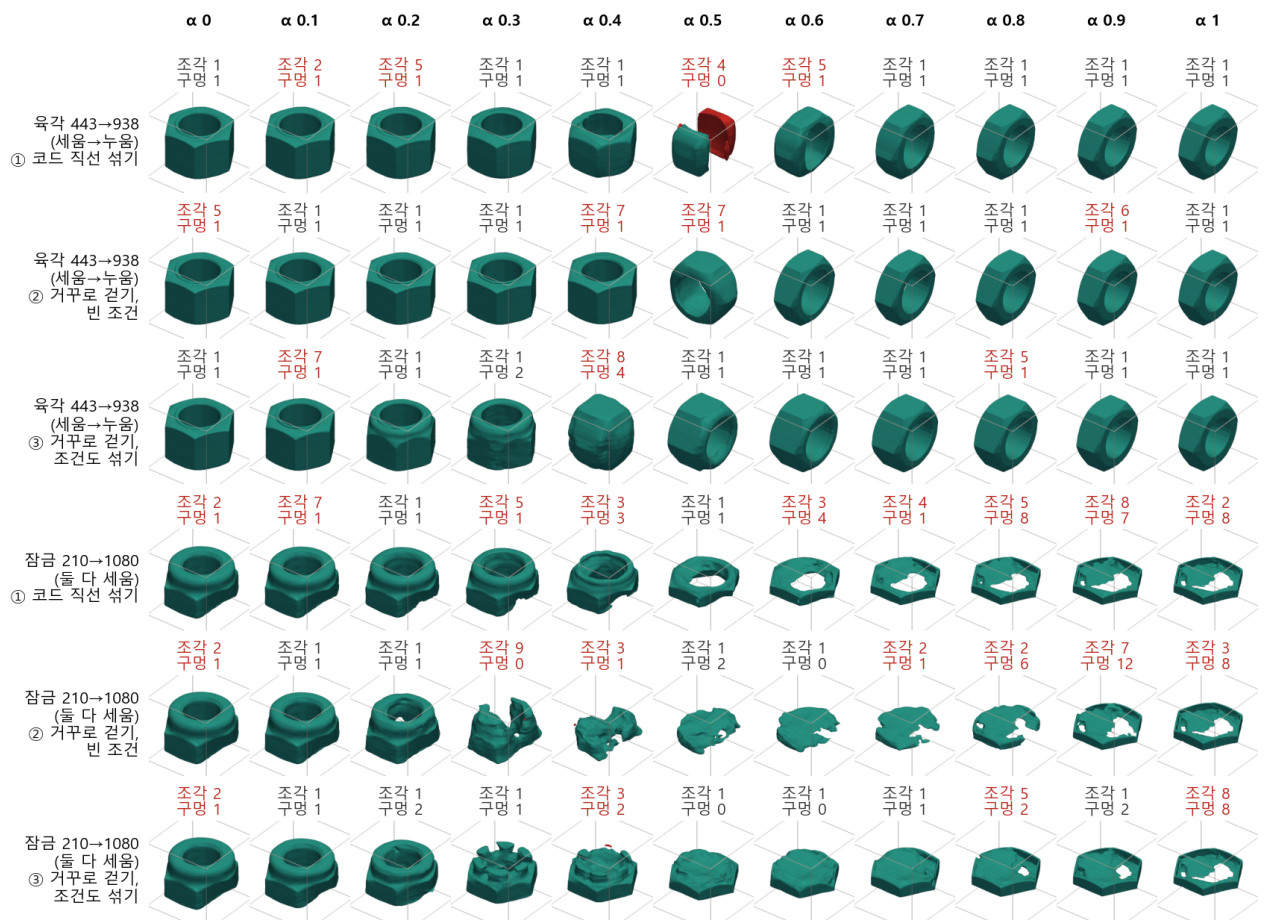


그림 18: $1e-8$ 모델, 같은 종류의 점검용 쌍 2개를 세 방법으로 섞었다. 위 3행은 육각 443(세움)→938(누움), 아래 3행은 잠금 210→1080(둘 다 세움). 각 3행은 위에서부터 (1) 코드 직선 섞기, (2) 거꾸로 걸기·빈 조건, (3) 거꾸로 걸기·조건도 옮김. 칸의 숫자는 비율, 조각 수(b), 구멍 수(g)이고, 빨간 글씨는 조각이 2개 이상인 칸이다.

그림 18의 육각 쌍은 세운 너트와 납힌 너트다. 세 방법 모두 비율 0.4-0.5에서 한 번에 넘어가며, 도중에 서서히 기울어지는 형상은 나오지 않는다. 조건도 옮기는 방법은 상자 크기 조건까지 직선으로 옮기므로 중간에 정육면체에 가까운 상자를 요구하고, 그래서 비율 0.4에서 덩어리(조각 8개, 구멍 4개)가 된다. 잠금 너트 쌍은 끝점(1080번)부터 구멍이 불규칙한 얇은 판이다.

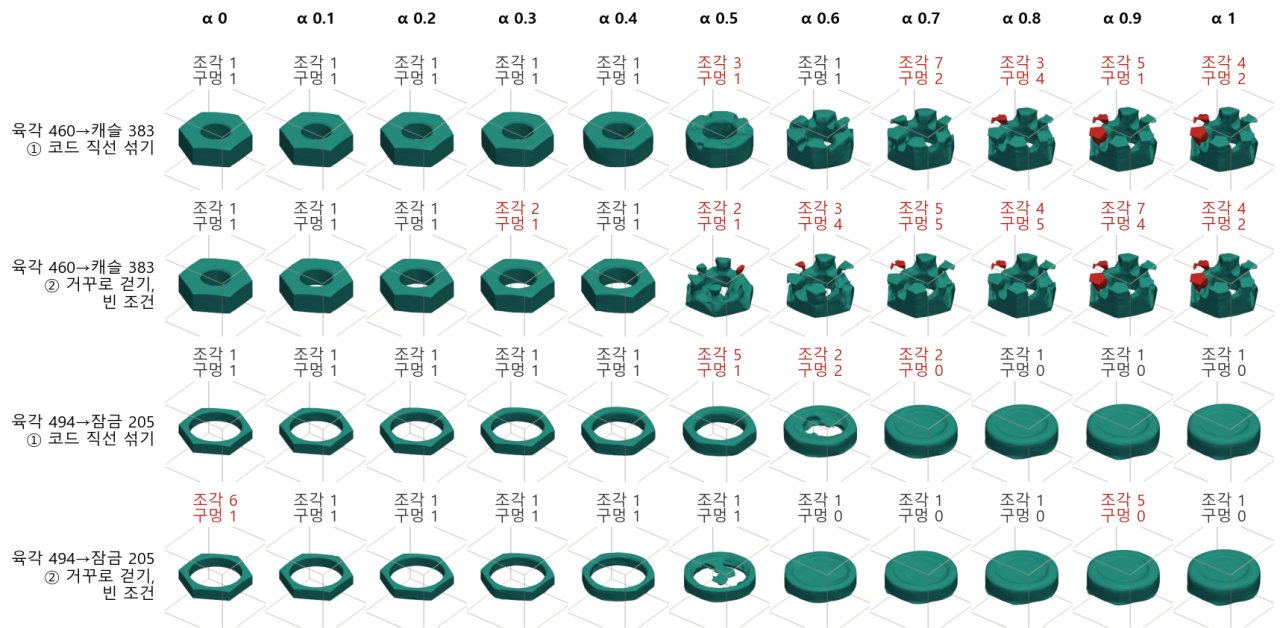


그림 19: 1e-8 모델, 다른 종류 쌍. 위: 육각 460→캐슬 383, 아래: 육각 494→잠금 205. 각각 코드 직선 쉼기와 거꾸로 걸기·빈 조건. 육각→캐슬 경로의 빨간 조각은 비율 1(끝점)에 이미 있다.

8.4 다른 종류 사이 쉼기

육각→캐슬 경로의 떨어진 조각은 쉼기가 만든 것이 아니다. 캐슬 383번을 복원한 끝점부터 조각나 있다 (그림 19). 육각 494→잠금 205는 두 끝의 방향이 같고(둘 다 세움), 비율 0.5–0.6에서 한 번 전환한 뒤로는 깨끗하다.

8.5 끝점 복원 확인

보간 경로는 끝점보다 좋아질 수 없다. 그림 20에서 캐슬 383번과 잠금 1080번은 쉼기 전, 1단계 복원에서 이미 실패한다. 잠금 1080번은 두께가 0.27(상자 1.8 기준 15%)인 얇은 판이다.

8.6 틱의 상당 부분은 방향 전환이다

§ 7.2에서 본 것처럼 ex3에는 같은 너트가 세워지거나 눕혀진 채로 쉼여 있다. 경로를 두 끝의 방향이 같은 쌍과 다른 쌍으로 나눠 보면 차이가 뚜렷하다(표 10). 두 끝 가운데 하나라도 비스듬하면 “다름”으로 섰다.

경로 묶음	두 끝 방향이 같음			두 끝 방향이 다름		
	경로 수	틱 중앙값	틱 > 3	경로 수	틱 중앙값	틱 > 3
SAME (같은 종류)	36	2.21	25%	36	5.09	75%
CROSS (다른 종류)	30	3.14	57%	18	5.39	83%
전체	66	2.52	39%	54	5.31	78%

표 10: 방향이 같은 쌍과 다른 쌍의 틱. 정상 모델 3개(1e-6, 1e-8, 1e-10)의 경로를 모두 합쳤다 (SAME은 쌍 4개 × 방법 3개 × 모델 3개, CROSS는 방법 2개).

방향이 다르면 틱 지수가 약 두 배가 된다(2.52 대 5.31). 모델은 세움, 누움(x), 누움(y)을 서로 다른 무리로 배웠고, 그 사이에는 서서히 기울어지는 형상이 데이터에 없으므로, 어떤 쉼기든 한 번에 뒤집힐 수밖에 없다. DeepJEB는 모든 형상의 방향이 같아서 이 문제가 없다.

육각 너트 443 육각 너트 938 잠금 너트 210 잠금 너트 1080 육각 너트 460 캐슬 너트 383 육각 너트 494 잠금 너트 205

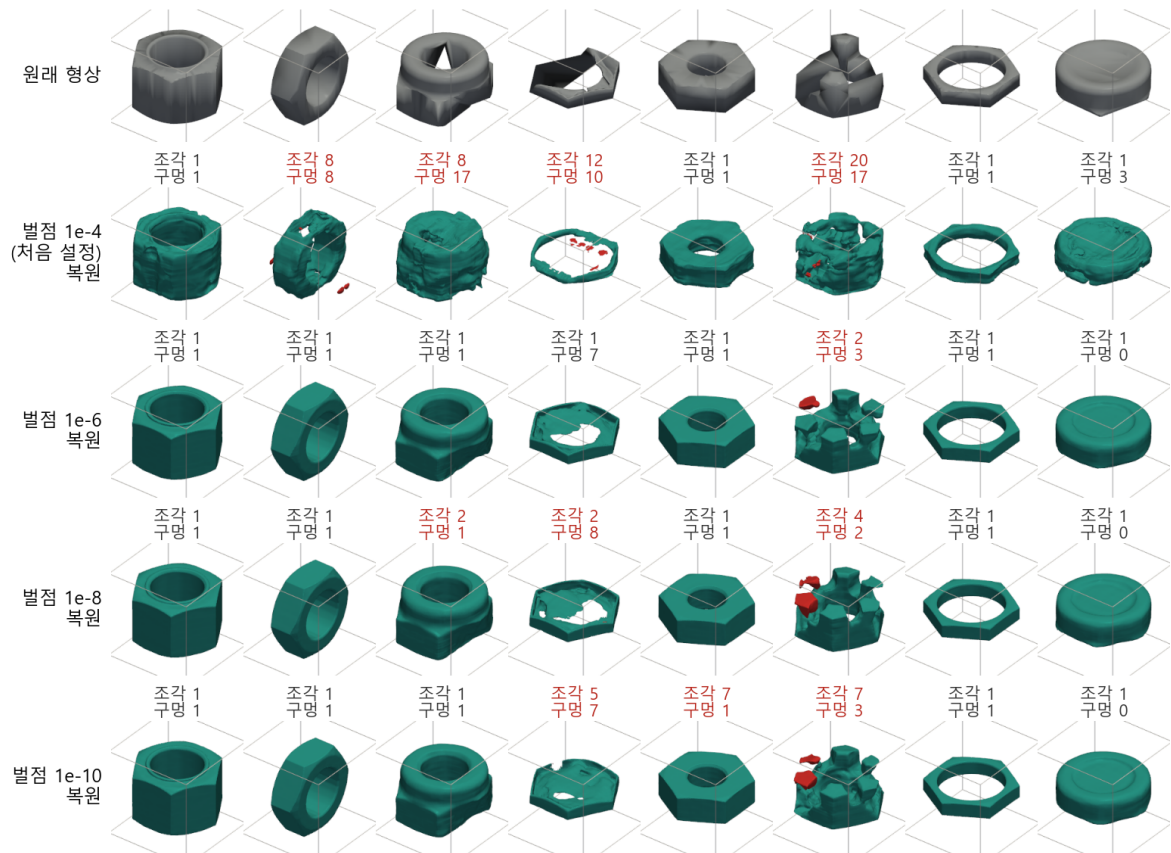


그림 20: 끝점으로 쓴 점검용 형상 8개의 정답(맨 위 행)과 모델 4개의 복원. 캐슬 383번은 모든 모델에서 떨어진 조각(빨강)이 있고, 잠금 1080번은 구멍 가장자리가 찢어져 있다. 처음 설정(1e-4)은 대부분 심하게 부서진다.

8.7 조건부로 하면 나아지나

“조건부로 한다”는 세 가지로 해석할 수 있어서 셋 다 돌렸다(표 11).

방식	비교	모델	틈 중앙값	틈 > 3	끝점 차이	가장 큰 한 번 변화
(a) 조건 하나로 고정하고 잡음만 섞기 (D1c)	빈 조건	1e-6	7.49	88%	0.71	0.72
	조건 고정	1e-6	2.44	25%	0.09	0.07
	빈 조건	1e-8	3.14	75%	0.73	0.61
	조건 고정	1e-8	2.36	0%	0.19	0.07
(b) 실제 형상 A→B, 조건도 같이 옮김 (SAME)	빈 조건	1e-6	3.28	63%	0.86	0.64
	조건도 옮김	1e-6	1.96	25%	0.86	0.49
	빈 조건	1e-8	4.70	63%	0.87	0.57
	조건도 옮김	1e-8	2.33	38%	0.87	0.48
(c) 잡음 고정, 조건만 A→B (CSWEEP)	강조 1	1e-6	2.69	38%	0.91	0.69
	강조 2	1e-6	2.15	38%	0.93	0.68
	강조 1	1e-8	2.29	38%	0.93	0.71
	강조 2	1e-8	2.73	50%	0.94	0.72

표 11: 조건을 쓰는 세 방식. 끝점 차이와 가장 큰 한 번 변화는 격자 내부의 1 – 겹침률, 경로별 중앙값이다. (c)는 이웃 비율 사이의 변화가 모두 커서(그 중앙값 약 0.3) 틈 지수가 작게 나온다. (c)는 틈 지수가 아니라 가장 큰 한 번 변화로 읽어야 한다.

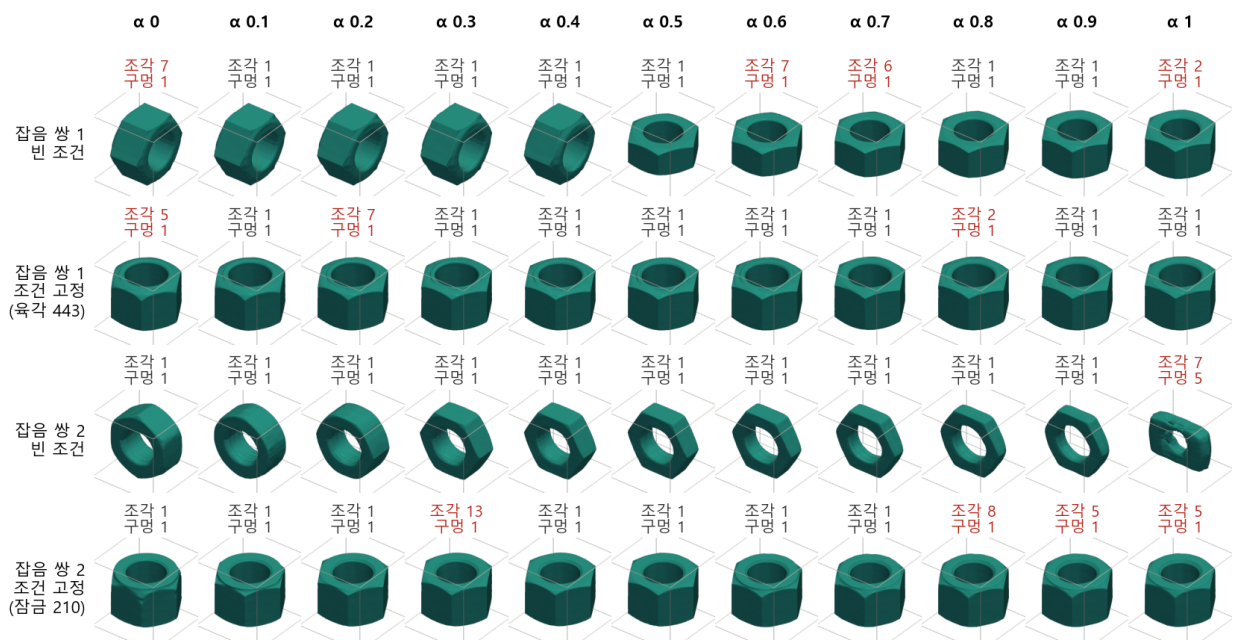


그림 21: 1e-6 모델, 같은 잡음 쌍 2개. 1행과 3행은 빈 조건, 2행과 4행은 조건 하나로 고정(육각 443, 잠금 210의 실제 조건). 빈 조건에서는 누운 너트가 세운 너트로 뒤집히지만, 조건을 고정하면 11칸이 거의 같은 너트다. 잠금 210의 조건을 줬는데 육각에 가까운 너트가 나왔다.

(a) 조건을 고정하면 틈이 사라지지만, 그것은 형상이 거의 움직이지 않기 때문이다. 조건을 고정하면 두 끝의 차이가 1 – 겹침률로 0.09–0.19밖에 안 된다(빈 조건은 0.71–0.73). ex3의 조건 5개가 형상을 거의 결정해 버리는 것이다. 게다가 상자 크기 조건에 방향이 들어 있다. 세운 육각 443은 상자가 (1.80, 1.56, 1.08), 누운 938은 (0.61, 1.80, 1.56)이다. 조건을 고정하면 방향도 고정되므로 방향 전환형 틈이 없어진다. 탐색용으로는 쓸모가 적다. 또 조건에 너트 종류가 없어서, 잠금 너트의 조건을 줘도 크기·부피·면적이 비슷한 가장 흔한 형상(육각)이 나온다(그림 21). 1e-6에서는 조건을 고정하자 추가 조각이

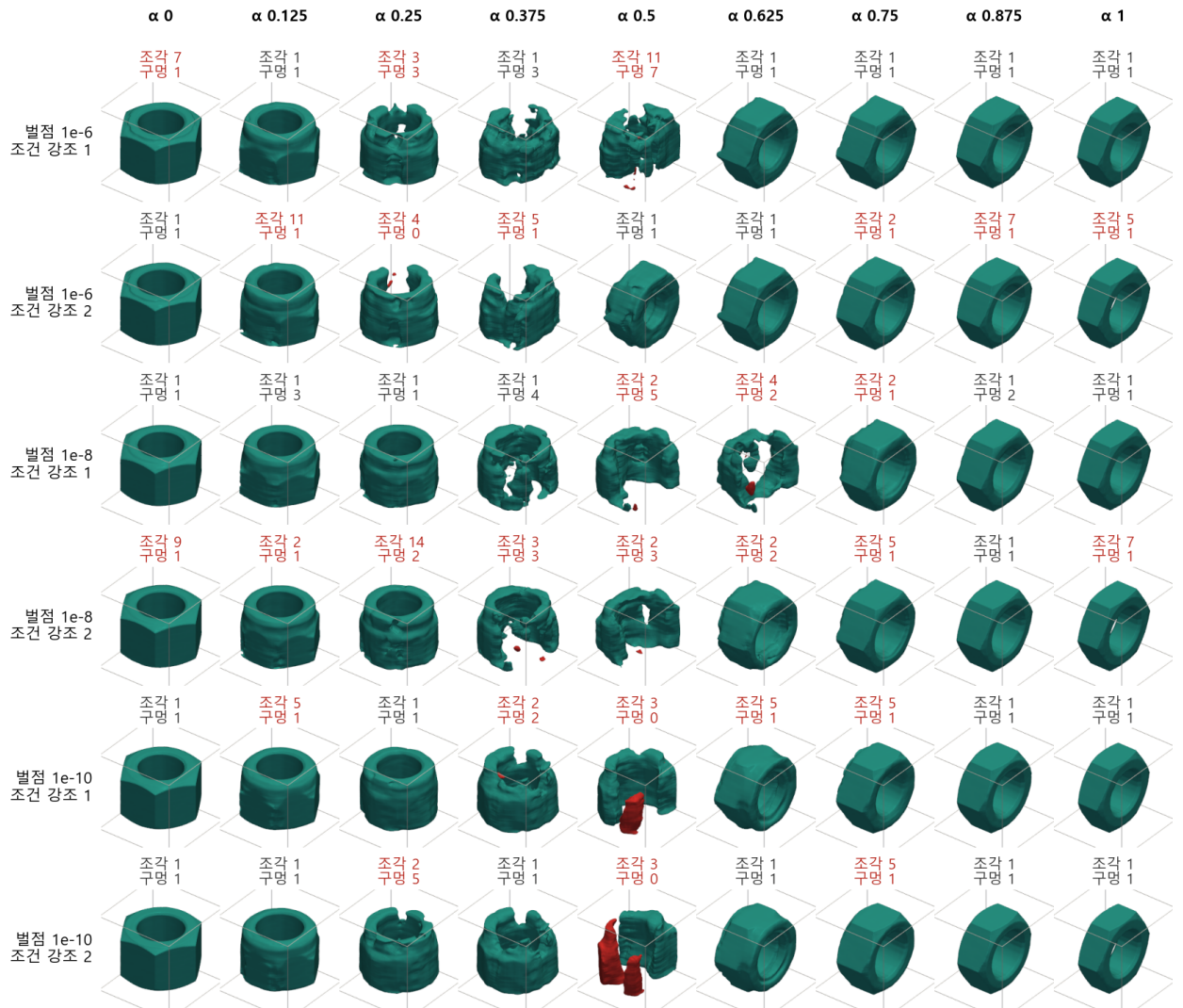


그림 22: 조건만 옮기기(CSWEEP), 육각 443(세움, 상자 $1.80 \times 1.56 \times 1.08$) → 육각 938(누움, $0.61 \times 1.80 \times 1.56$), 잡음 1개. 행은 1e-6 / 1e-8 / 1e-10 모델 × 강조 배율 1 / 2, 열은 비율 9칸. 두 끝은 실제 형상과 잘 맞지만, 비율 0.25–0.5에서는 거의 모든 행에 너덜너덜한 긴 통이 나온다. 칸 위 숫자는 비율, 조각 수(b), 구멍 수(g)다.

오히려 늘었다(0.097 → 0.403).

(b) 실제 형상 사이 섞기에는 도움이 된다. 이웃 비율 사이 변화 가운데 가장 큰 것이 0.57–0.64에서 0.48–0.49로 줄어, 변화가 여러 구간에 나뉜다. 대신 돌아가는 경로가 되고 (경로 길이 / 직선 거리 2.3–2.4 → 2.9), 구멍 수 오르내림은 그대로다.

(c) 조건만 옮기면 조건은 잘 따르지만 형상은 매끄럽지 않다. 조건 오차는 표 12와 같다. 상자 크기는 5% 안, 면적은 7–11%, 부피는 8–18% 안에서 맞고, 부피를 크게 바꾸라고 하면 순서도 거의 완벽하게 지킨다 (요구 변화가 155–178%인 잠금·캐슬 쌍의 부피 순위 상관 0.90–1.0). 요구 변화가 19%뿐인 홈 너트 쌍은 –0.33에서 0.52 사이로 잡음에 묻힌다. 그러나 가장 큰 한 번 변화가 0.68–0.72로 세 방식 가운데 가장 크다. 육각·홈 쌍은 두 끝의 방향이 다르고 (그림 22), 잠금 1080번은 얇은 판이라 잠금 쌍 경로 12개 (모델 3개 × 잡음 2개 × 강조 배율 2개) 중 6개가 비율 0.875–1에서 19–364조각으로 쪼개지며, 캐슬은 전 구간이 여러 조각이다. 비율 0.5의 상자(1.20 × 1.68 × 1.32)는 데이터에 드물다. 세 치수가 모두 10% 안에 드는 형상은 1305개 중 6개(0.5%) 뿐이고, 세운 육각 443의 상자 근처에는 136개(10%)가 있다.

모델	강조	상자 x/y/z	부피	면적	5개 평균	부피 순위 상관	중간 한 조각
1e-4 (처음)	1	6.7 / 1.8 / 15.6%	24.6%	17.0%	14.2%	1.00	36%
1e-4 (처음)	2	7.9 / 2.3 / 11.5%	20.6%	23.6%	16.2%	0.99	21%
1e-6	1	0.7 / 2.9 / 5.0%	15.8%	10.8%	8.3%	0.94	77%
1e-6	2	0.7 / 2.9 / 3.0%	11.2%	7.1%	6.4%	0.93	59%
1e-8	1	1.0 / 2.9 / 4.1%	18.0%	9.3%	8.6%	0.96	73%
1e-8	2	0.5 / 2.5 / 3.2%	8.1%	6.8%	5.8%	0.83	54%
1e-10	1	0.7 / 2.7 / 3.8%	15.7%	6.6%	7.1%	0.94	71%
1e-10	2	1.0 / 2.5 / 2.5%	11.5%	6.9%	6.1%	0.87	64%

표 12: 조건만 옮기기(CSWEEP)의 조건 오차(경로별 중앙값), 부피 순위 상관, 중간 비율의 한 조각 비율(평균). 경로는 모델·배율마다 8개.

강조 배율 2는 얻는 것과 잃는 것이 함께 있다. 부피 오차는 16–18%에서 8–12%로 줄지만, 중간 한 조각 비율이 71–77%에서 54–64%로 떨어진다. 형상 품질이 우선이면 1, 치수를 정확히 맞춰야 할 때만 2를 권한다.

조건이 형상을 결정한다는 말의 뜻. “그 조건의 전형적인 형상이 나온다”는 뜻이다. 실제 형상의 조건을 주고 임의의 잡음으로 만든 CSWEEP 끝 형상은 그 실제 형상과 겹침률이 0.63–0.72다(캐슬 제외 0.73–0.76). 육각 쌍의 끝(443, 938)은 0.88–0.98이지만 잠금 1080번은 0.18–0.52, 캐슬 355번은 0.01–0.18이다. 흔한 형상일수록 잘 맞는다.

캐슬은 조건으로 해결되지 않는다. 캐슬 조건을 준 D1c 경로 2개 가운데 캐슬 19번 조건 경로는 모든 비율에서 3–12조각이고, 캐슬 383번 조건 경로도 11칸 중 6–8칸이 여러 조각이다(1e-6, 1e-8 모두). 원인은 1단계 복원이다(§ 8.8).

8.8 캐슬 너트와 1단계 점검용 오차

- 대부분은 괜찮고, 일부 종류(잠금·캐슬)가 나쁘다. 학습 기록의 점검용 오차는 형상 평균이라 오차가 큰 형상 몇 개가 평균을 끌어올린다(표 13).
- 캐슬은 정답 자체가 어렵다. 정답 부호 거리를 같은 128³ 격자로 뽑아도 한 조각 비율이 66.7%다. 그림 20의 빨간 조각 가운데 일부는 격자 해상도 탓이다.

지표 (kl 1e-6 모델)	값
요약부로 복원한 표면 평균 거리 중앙값: 점검용 / 학습용	0.0069 / 0.0069
부호 거리 오차 평균의 비율 (점검용 / 학습용)	3.6배 (중복 형상을 빼면 4.7배)
종류별 오차 비율: 육각 / 잠금 / 캐슬	1.05배 / 2.4배 / 10배
정답 형상을 같은 격자로 뽑았을 때 한 조각 비율: 전체 / 캐슬	90.8% / 66.7%

표 13: 1단계 점검용 성능. 대부분의 형상은 학습용과 같은 품질로 복원되고, 평균을 끌어올리는 것은 오차가 유난히 큰 소수의 잠금·캐슬 너트다.

- 원인은 종류다. 1e-6에서 실제로 떨어진 조각(격자 칸 100개 초과)이 나온 점검용 형상 6개는 모두 캐슬이었다. 우연히 이렇게 몰릴 확률은 0.03%다. 벽 두께와는 관계가 없었다(순위 상관 -0.02).
- 복원부보다 요약부 쪽이다. 코드를 직접 고치면(`latent_refine_steps 1000`) 정답 격자 수준의 1.04-1.09배까지 좋아진다.
- 점검용 수치는 오히려 후하게 나온 것이다. ex3는 `split_by_parent False`라, 점검용 130개 가운데 27개가 학습용 형상 하나와 조건 5개가 완전히 같다. 확인한 8쌍 중 7쌍은 모양도 같은 중복이었다.

8.9 조건 없는 생성 결과

	1e-4 (처음)	1e-6	1e-8	1e-10	정답 격자 상한
조건 없이 32개 생성, 한 조각 비율	37.5%	81.3%	81.3%	90.6%	90.8%
실제 조건 + 강조 2, 5개 조건 평균 오차	8.15%	1.68%	1.62%	1.41%	—

표 14: 2단계 생성 결과. 1e-10은 조건 없는 생성의 한 조각 비율이 정답을 같은 격자로 뽑았을 때와 거의 같다. CSWEET(표 12)의 오차가 이보다 큰 것은, CSWEET이 데이터에 드문 중간 조건을 요구하기 때문이다.

9 DeepJEB

9.1 왜 DeepJEB로 먼저 하나

ex3의 문제 가운데 두 가지(방향이 섞여 있음, 캐슬처럼 수가 적고 어려운 종류)는 데이터 쪽 문제다. DeepJEB는 둘 다 없다. 모든 형상의 방향이 같고, 종류가 브래킷 하나이며, 부모 형상 263개의 형제들이 “같은 계열의 연속 변형”을 이룬다. 그래서 DeepJEB에서

- 보간이 매끄러우면 → ex3의 문제는 데이터(방향, 캐슬) 탓으로 확정되고,
- 구멍 수 오르내림이 여전히 남으면 → 1단계가 형상들을 형상 코드로 늘여놓은 방식 자체의 문제다.

9.2 J0: 기존 Studio 실험은 붕괴했다

사용자가 Studio에서 돌린 J0(aarl GPU 3, 대표점 512, kl 1e-6, 1단계 500바퀴)는 흔들림을 키우는 50-150바퀴 구간부터 살아 있는 칸이 빠르게 줄어, 16384개 가운데 50바퀴 15441개에서 150바퀴 1518개, 160바퀴 564개, 295바퀴 7개가 되었다(그림 9). ex3 처음 설정과 같은 모양이다. 곱이 1.6×10^{-2} 로 ex3 정상 모델(10^{-3})보다 16배 세다. 점검용 오차는 75바퀴에 0.00228로 가장 낮았고, 160바퀴에 0.0223까지 올랐다가 295바퀴에 0.00473이었다. J0의 기록은 295바퀴까지만 확보했다. J0는 사용자 작업이라 건드리지 않았다.

실험	T	kl_weight	곱	바퀴	점검용 오차 (바퀴)	살아 있는 칸	1단계 끝난 시각
J1 (GPU 0)	512	$1e-8$	1.6×10^{-4}	500 / 500	$1.03e-3$ (495)	16384 / 16384	9/26 15:33
J2 (GPU 1)	512	$1e-10$	1.6×10^{-6}	500 / 500	$1.04e-3$ (499)	16384 / 16384	9/26 15:53
J3 (GPU 2)	32	$1e-8$	1×10^{-5}	500 / 500	$2.64e-3$ (495)	1024 / 1024	9/26 06:22

표 15: 최종 결과. 점검용 오차는 가장 낮은 값과 그 바퀴다. 1단계 한 바퀴에 J1 약 130초, J2 약 140초, J3 약 64초가 걸렸다. 1단계가 끝나면 2단계(500바퀴)가 자동으로 이어졌고, J3는 약 13분(06:36), J1은 약 19분(15:53), J2는 약 21분(16:14)에 끝났다. 세 실험 모두 끝까지 모든 칸이 살아 있었다. GPU 번호는 aarl 기준이다.

9.3 J1-J3 1단계 결과

무너짐 판정 기준. 50-160바퀴 사이에 점검용 오차가 60바퀴까지의 최저값의 2배를 넘거나, 살아 있는 칸이 전체의 절반 아래로 떨어지면 “무너짐”으로 본다. J0는 160바퀴에 오차가 0.0223으로 60바퀴까지의 최저값(0.00242)의 약 9배, 살아 있는 칸이 564개(3%)여서 두 기준을 모두 넘었다.

J3: 무너지지 않음. J3는 50-160바퀴 내내 오차가 최저값의 1.09배를 넘지 않았고, 살아 있는 칸은 50바퀴에 694개(68%)로 가장 적었다가 80바퀴부터 1024개 전부가 살아 있다. 점검용 오차는 계속 내려가 340바퀴에 $2.68e-3$ 이다. 300바퀴 뒤로는 거의 평탄하다.

J1·J2: 무너지지 않음. 50-160바퀴 동안 점검용 오차는 60바퀴까지의 최저값의 1.12배(J1), 1.11배(J2)를 넘지 않았다. 이 가장 높은 값은 흔들림을 섞기 시작한 50바퀴 바로 그때의 값이고, 그 뒤로는 계속 내려갔다. 살아 있는 칸도 같은 50바퀴에 J1 14426개(88%), J2 14207개(87%)로 가장 적었다가 다시 늘어 160바퀴에는 둘 다 16384개 전부가 살아 있다. 같은 160바퀴에서 J0는 살아 있는 칸이 564개(3%), 점검용 오차가 0.0223이었다. J1·J2는 같은 바퀴에 오차가 $1.59e-3$, $1.61e-3$ 으로 J0의 약 14분의 1이다. 대표점 수와 구조는 J0와 같고 kl_weight 만 100배(J1), 1만 배(J2) 작으므로, J0가 무너진 까닭은 KL 무게가 너무 컸기 때문이라고 볼 수 있다. J1과 J2의 차이는 끝까지 거의 없었다(500바퀴 끝 $1.03e-3$ 대 $1.04e-3$, 1.4% 차이).

J4(대표점 512, $kl\ 6.25 \times 10^{-8}$, $ex3\ 1e-6$ 과 같은 곱)는 제안만 했고 돌리지 않았다. GPU 3이 되면 J0 대신 돌릴 수 있지만, 결정은 사용자에게 있다.

9.4 계획과 진행

1. **1단계 판정.** 50-250바퀴 구간에서 살아 있는 칸과 점검용 오차가 $ex3$ 정상 모델처럼 유지되는지 본다. 끝남: 셋 다 무너지지 않았다.
2. **2단계.** 1단계에 이어 자동으로 500바퀴 학습된다. $ex3$ 처럼 500바퀴가 모자란지 가장 좋은 지점의 위치를 본다. 끝남: 셋 다 가장 좋은 지점이 495바퀴 이후다.
3. **보간 평가.** $ex3$ 와 같은 평가 프로그램을 DeepJEB에 맞게 돌린다. DeepJEB의 묶음은 종류가 아니라 부모 형상이므로 SAME = 같은 부모의 형제, CROSS = 다른 부모로 나눈다. 같은 계열 안의 연속 변형과 계열 사이 이동을 따로 볼 수 있다. 부분 진행: 평가는 돌렸지만 쌍을 부모 기준으로 나누지는 않았다.
4. **조건 제어.** 조건이 부피, 면적 둘뿐이므로 부피를 요청한 만큼 따라가는지 조건만 옮기기로 본다. 아직 하지 않았다.
5. 결과를 그림 9에 겹쳐 그리고 아래에 채웠다.

데이터 밀도. DeepJEB는 형상당 부호 거리 점이 10240개($ex3$ 의 1/8)뿐이다. `num_query_points` 8192면 매 바퀴 거의 같은 점을 다시 보게 되어 얇은 부분을 덜 배우고 학습용 형상을 외우기 쉽다. J1-J3의 점검용 오차가 학습용과 크게 벌어지면, 원본 (D:\CAE_datasets_raw\)에서 형상당 약 8만 점으로 다시 만드는 것이 다음 수다. 실제로는 크게 벌어지지 않았으므로(아래 결과) 지금은 필요하지 않다.

9.5 결과

- 1단계 판정(살아 있는 칸, 점검용 오차, 가장 좋은 바퀴): J3는 500바퀴 끝까지 1024칸이 모두 살아 있었고, 점검용 오차 $2.64e-3$ (495바퀴)으로 끝났다. 340바퀴($2.68e-3$) 뒤로는 1.4% 정도만 더 내려갔으므로 500바퀴면 충분했다. J1·J2도 500바퀴 끝까지 16384칸이 모두 살아 있었고, 점검용 오차 $1.03e-3$ (J1, 495바퀴), $1.04e-3$ (J2, 499바퀴)으로 끝났다. 400바퀴 뒤로는 1.5% 안팎만 더 내려갔으므로 역시 500바퀴면 충분했다.
- 점검용 오차와 학습용 오차의 차이(형상당 점을 더 많이 다시 만들지 판단): J3는 마지막에 학습용 $1.88e-3$, 점검용 $2.64e-3$ 으로 점검용이 1.4배다. ex3의 약 3배보다 작아서, 점이 적어 학습용 형상을 외우는 문제는 J3에서는 크지 않다. J1·J2는 255바퀴에 학습용이 점검용보다 오히려 크다(J1 $1.50e-3$ 대 $1.26e-3$, J2 $1.60e-3$ 대 $1.32e-3$. 학습용은 흔들림을 섞고 재기 때문이다). 500바퀴 끝에도 학습용이 조금 크다(J1 $1.08e-3$ 대 $1.03e-3$, J2 $1.09e-3$ 대 $1.04e-3$). 세 실험 모두 점검용이 학습용보다 크게 벌어지지 않았으므로, 형상당 점을 다시 만들 필요는 지금은 없다.
- 2단계 점검용 오차와 가장 좋은 바퀴: J3는 0.834(475바퀴, 학습용 0.763). 400바퀴 뒤로는 0.1% 안쪽으로만 움직였다. J1은 0.744, J2는 0.729(둘 다 495바퀴)로 끝났다. 둘은 끝날 때까지 꾸준히 내려가고 있어서(J1은 220바퀴 0.763 → 495바퀴 0.744) 500바퀴가 모자랐을 수 있다. 형상 코드의 숫자 개수가 달라 이 오차 값을 J3와 직접 비교하지는 않는다.
- 조건 없는 생성의 한 조각 비율, 조건 오차: J3는 조건 없이 72%, 실제 조건을 주면 100%가 한 조각이고, 부피·면적 오차 중앙값은 둘 다 2.5%다(아래 § 9.6, 표 16). J1·J2는 조건 없이 31%, 22%, 실제 조건을 주어도 38%, 28%만 한 조각이고, 부피 오차 중앙값이 8.3%, 7.3%로 J3의 약 3배다.
- 보관 통계: J3는 실제 형상 쌍의 거꾸로 걸기 섞기에서 중간 형상의 96%가 한 조각이다(§ 9.6). J1·J2는 같은 섞기에서 54%, 58%다. SAME(형제)과 CROSS(다른 부모) 구분: **[결과 대기: 평가 프로그램이 쌍을 부모 기준으로 고르도록 바꾼 뒤]**
- 부피 조건 옮기기(순위 상관, 조건 오차): **[결과 대기: 아직 돌리지 않음]**
- 결론은 § 9.6 끝에 있다. 1단계가 무너지지 않으면 DeepJEB의 섞기는 부스러기 없이 이어진다(J3). 대표점을 512개로 늘리면 복원은 좋아지지만, 같은 2단계 학습량에서는 생성과 섞기가 나빠진다. 2단계 처방을 합치면 J3 수준까지 오른다(§ 10). 구멍 수와 형제/다른 부모 구분을 보지 않았으므로 ex3의 문제가 데이터 탓인지는 아직 확정하지 않는다.

9.6 생성·섞기 평가 (J3, J1, J2, J0)

두 단계를 모두 마친 J3, J1, J2와 1단계가 무너진 J0(사용자 Studio 실험의 마지막 모델)를 같은 절차로 평가했다(그림 23). 형상 쌍, 조건, 격자 크기를 네 모델에 똑같이 썼으므로 차이는 모델에서만 온다.

읽는 법. 한 조각 비율은 128칸 격자에서 표면을 뽑았을 때 떨어진 조각이 없는 형상의 비율이다. 두 가지를 먼저 알아 둔다.

- 여러 조각으로 나온 형상도 100칸보다 큰 조각은 거의 하나뿐이었다(묶음별 평균 1.00–1.13개. 1을 넘는 것은 잡음 쌍의 J1, J2, J0뿐). 즉 여기서 “여러 조각”은 본체 옆에 붙은 작은 부스러기이며, 형상이 두 동강 난 경우는 드물다. 부스러기의 부피는 형상 전체의 0.1% 아래다.
- 실제 점검용 형상 16개를 형상 코드에서 그대로 복원해도 81–88%만 한 조각이다(J3, J1, J0 81%, J2 88%). 1단계 복원에서 이미 부스러기가 생기므로 생성 결과의 한 조각 비율도 이 값 근처에서 비교해야 한다.

평가 개수가 24–48개로 적어서 형상 하나가 2–4%에 해당한다. 몇 % 차이는 우연일 수 있다.

조건 따르기와 베끼기. 실제 부피·면적을 조건으로 주면 J3는 32개 모두 한 조각이고, 요청값과의 차이가 중앙값 2.5%다. J0는 부피 차이가 11.7%로 약 5배 크다. J3는 학습 형상을 베낀 것이 하나도 없는

DeepJEB 생성·보간 평가 절차

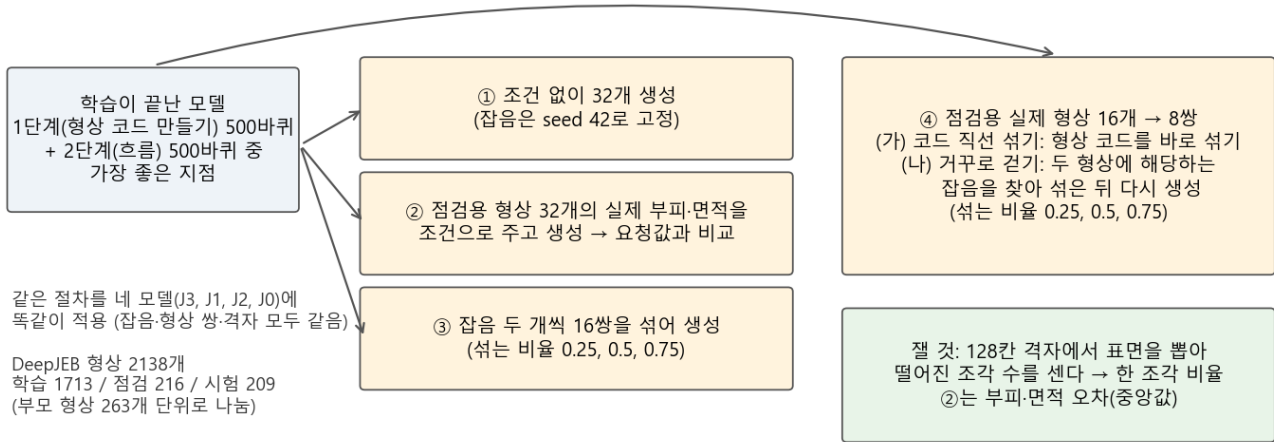


그림 23: DeepJEB 평가 절차. 네 가지 모두 가장 좋은 지점의 모델로 50걸음을 생성한다. ④의 (가)는 두 실제 형상의 형상 코드를 곧바로 섞어 복원하고, (나)는 생성 과정을 거꾸로 걸어 두 형상에 해당하는 잡음을 찾은 뒤 그 잡음을 섞어 다시 앞으로 걷는다(그림 16의 (나), (다)와 같다). 점검용 형상은 학습에 쓰지 않은 부모 형상 26개의 형제들이다.

항목	개수	J3	J1	J2	J0
대표점, kl_weight		32, 1e-8	512, 1e-8	512, 1e-10	512, 1e-6
조건 없이 생성, 한 조각	32	72%	31%	22%	72%
실제 조건으로 생성, 한 조각 (강조 배율 1)	32	100%	38%	28%	94%
부피 오차 중앙값	32	2.5%	8.3%	7.3%	11.7%
면적 오차 중앙값	32	2.5%	4.0%	3.1%	4.7%
실제 조건으로 생성, 한 조각 (강조 배율 2)	32	97%	50%	50%	94%
부피 오차 중앙값	32	4.3%	11.9%	13.3%	13.0%
학습 형상을 거의 그대로 베낀 비율	32	0%	(9.4%)	(75%)	9.4%
조건 없는 생성의 부피 퍼짐(표준편차)	32	0.078	0.038	0.037	0.041
잡음 16쌍: 양 끝(섞지 않음), 한 조각	32	66%	16%	31%	75%
잡음 16쌍: 중간 3개, 한 조각	48	67%	15%	25%	79%
잡음 3쌍, 9단계, 한 조각	27	59%	15%	7%	93%
실제 형상 8쌍, 코드 직선 섞기 중간, 한 조각	24	88%	79%	83%	79%
실제 형상 8쌍, 거꾸로 걷기(빈 조건) 중간, 한 조각	24	96%	54%	58%	71%
실제 형상 8쌍, 거꾸로 걷기(조건도 옮김) 중간, 한 조각	24	92%	63%	58%	75%
거꾸로 걷기 되돌림의 형상 코드 차이(중앙값)	16	0.01%	0.10%	0.10%	0.37%
참고: 실제 형상 16개를 그대로 복원, 한 조각	16	81%	81%	88%	81%

표 16: 네 모델의 평가 수치. J0만 1단계가 무너진 모델이다. “베낀 비율”은 생성 형상의 형상 코드가 가장 가까운 학습 형상과, 점검용 형상과 학습 형상 사이 가장 가까운 거리의 하위 5%보다 더 가까운 비율이다(J0는 32개 중 3개. 그중 1개는 점검용 형상의 가장 가까운 거리보다도 가깝다). 부피 퍼짐은 크기를 맞춘 부피의 표준편차이며 클수록 다양하다. J1·J2의 베낀 비율(괄호)은 믿을 수 없다. 숫자가 16384개이면 형상 코드 사이 거리가 모두 비슷해져(J2의 점검용 형상과 가장 가까운 학습 형상의 거리가 1.335–1.372) 하위 5% 기준선이 거의 중앙값과 같아지기 때문이다.

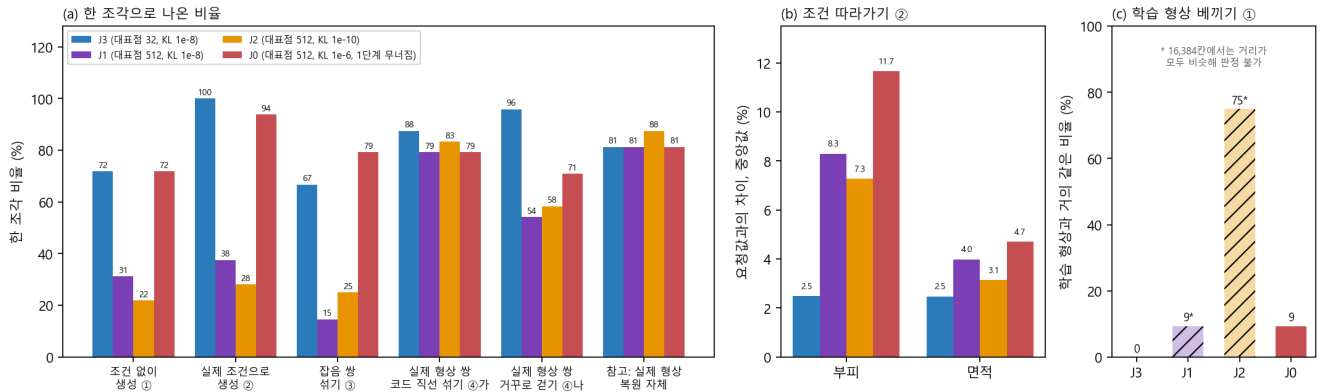


그림 24: 표 16의 주요 수치. (a) 한 조각 비율, (b) 실제 조건을 줄 때 요청한 부피·면적과의 차이, (c) 학습 형상을 베끼 비율. 번호 ①-④는 그림 23의 절차 번호다. (c)의 J1·J2(빛금)는 판정할 수 없는 값이다(표 16 설명).

반면 J0는 32개 중 3개가 학습 형상과 거의 같다. 조건 없는 생성의 부피 퍼짐도 J3가 J0의 약 2배다. 1 단계가 무너지면 형상 코드에 담긴 정보가 줄어, 생성이 적은 수의 학습 형상 쪽으로 몰린다고 볼 수 있다. 강조 배율을 2로 올리면 J3의 부피 차이가 4.3%로 오히려 커지므로, DeepJEB에서는 배율 1이 낫다.

실제 형상 사이 섞기. 실제 점검용 형상 두 개 사이를 거꾸로 걸기로 섞으면 J3는 중간 형상 24개 중 23개(96%)가 한 조각이다. 이는 실제 형상을 그대로 복원했을 때(81%)보다도 높다. 거꾸로 걸기 자체도 정확해서, 찾은 잡음으로 다시 앞으로 걸으면 형상 코드 차이가 중앙값 0.01%, 겹침률이 0.99997이다(J0는 0.37%, 0.9991). J0는 같은 섞기에서 71%로, 섞는 도중에 부스러기가 더 생긴다. 그림 25에 J3의 섞기 두 쌍과, 같은 쌍을 J1과 J0로 섞은 결과를 실었다.

잡음 섞기는 J0가 더 높게 나온다. 새 잡음 두 개를 섞는 경우에는 J3가 67%로 J0(79%)보다 낮다. 하지만 J3는 섞지 않은 양 끝 자체가 66%라서, 섞는다고 나빠지지는 않는다. 차이는 조건 없이 생성한 형상 자체의 부스러기 비율에서 온다. 잡음 3쌍 9단계(27개)에서는 J3 59%, J0 93%로 차이가 더 크지만 쌍이 셋뿐이다. 쌍별로 보면 16쌍 중 중간 셋이 모두 한 조각인 쌍이 J3 7쌍, J0 10쌍이고, 모두 여러 조각인 쌍이 J3 2쌍, J0 1쌍이라 쌍에 따라 차이가 크다. J0가 높은 까닭은 다양성이 낮아(부피 퍼짐 절반) 어느 잡음에서 출발해도 비슷하고 무난한 형상이 나오기 쉽기 때문이라고 추정한다.

대표점 512의 생성은 나빠졌다. 1단계가 무너지지 않은 J1·J2는 복원 오차가 J3의 2.6분의 1이고, 실제 형상을 그대로 복원하면 J3만큼 한 조각이 나온다(81%, 88%). 그러나 2단계로 만든 형상은 크게 나빠졌다. 조건 없이 만들면 31%, 22%만 한 조각이고(J3 72%), 잡음 쌍 끝점 기준으로 한 형상에 떨어진 조각이 평균 약 3개(J3 약 1.7개)다. 실제 조건을 주어도 38%, 28%만 한 조각이고 부피 오차가 J3의 약 3배다. 섞기에서도 2단계를 거치는 방법(잡음 섞기, 거꾸로 걸기)은 한 조각 비율이 J3보다 크게 낮다(거꾸로 걸기 54%, 58% 대 96%). 반면 2단계를 거치지 않는 코드 직선 섞기는 79%, 83%로 J3(88%)와 비슷하다. 거꾸로 걸기의 되돌림 차이도 0.1%로 작아서 잡음을 찾는 계산 자체의 문제도 아니다. 즉 나빠진 곳은 2단계 생성 모델이다. 2단계가 만들어야 할 숫자는 16배인데, 학습 형상 수(1713개)와 학습량(500바퀴, 약 20분)은 같았다. 2단계 점검용 오차가 끝까지 내려가던 것을 보면 학습이 모자랐을 가능성을 먼저 보았지만, 2000바퀴로 오래 학습해도 거의 그대로였다(§ 10). J1(kl 1e-8)과 J2(kl 1e-10)는 비슷하므로 kl을 1e-8 아래로 더 줄일 이유는 없다.

결론. 1단계가 무너지지 않은 J3는 무너진 J0보다 조건을 약 5배 정확히 따르고, 학습 형상을 베끼지 않으며, 더 다양하다. 실제 형상 사이 섞기도 부스러기 없이 이어진다. 대표점을 512개로 늘린 J1·J2는 복원만 좋아지고, 같은 2단계 학습량에서는 생성과 섞기가 J3보다 크게 나쁘다. 따라서 지금 DeepJEB에는 대표점 32개, kl 1e-8(J3)이 가장 낫다(2단계 처방으로 512와 128을 다시 본 결과는 § 10). 남은 약점은 조건 없는 생성의 약 30%에 붙는 작은 부스러기다. 이번 평가는 구멍 수를 세지 않았고 형제 쌍과 다른 부모 쌍도 나누지 않았으므로, ex3의 문제가 데이터 탓인지는 아직 확정하지 않는다.

J3, 형상 쌍 1, 코드 직선 섞기 ④가



J3, 형상 쌍 1, 거꾸로 걷기 ④나



J1 (대표점 512), 형상 쌍 1, 거꾸로 걷기 ④나



J3, 형상 쌍 0, 거꾸로 걷기 ④나



J0, 형상 쌍 0, 거꾸로 걷기 ④나



왼쪽 끝과 오른쪽 끝(파랑): 두 실제 형상의 복원. 가운데 셋(주황): 섞는 비율 0.25, 0.5, 0.75

그림 25: 실제 점검용 형상 쌍 사이의 섞기. 각 줄의 양 끝(파랑)은 두 실제 형상의 복원, 가운데 셋(주황)은 섞는 비율 0.25, 0.5, 0.75의 결과다. 위 세 줄은 같은 쌍 1을 J3의 코드 직선 섞기, J3의 거꾸로 걷기, J1의 거꾸로 걷기로 섞은 것이고, 아래 두 줄은 쌍 0을 J3와 J0로 거꾸로 걸어 섞은 것이다.

10 대표점 수 다시 보기: 32는 너무 작은가, 512는 안 되는가

질문. 3차원 형상 생성 논문들은 대표점을 512개 이상 쓰는 경우가 많다. 그런데 앞 절에서는 대표점 32개(J3)가 512개(J1)보다 생성이 훨씬 좋았다. 32가 너무 작은 것은 아닌지, 512를 제대로 쓰는 방법은 없는지를 2026년 9월 26-28일 aarl GPU 0-2에서 세 데이터로 다시 확인했다(GPU 3은 쓰지 않았다).

논문들과 무엇이 다른가. 대표점 512개 이상을 쓰는 형상 생성 연구는 학습 형상이 수십만~수백만 개다. 여기의 DeepJEB은 학습 형상이 1713개, MCB(ex3)는 1044개, DrivAerML(ex2)은 약 310개다(그림 26의 (a)). 2단계 생성 모델은 이 형상 수만큼의 점만 보고 숫자 $T \times 32$ 개짜리 공간의 분포를 배워야 한다. $T = 32$ 이면 숫자가 1024개로 형상 수와 비슷하지만, $T = 512$ 이면 16384개로 형상 수의 약 10-50배다. 형상 코드를 크게 만들면 복원은 좋아지고 생성은 어려워진다는 보고가 있고 (Yao 외, “Reconstruction vs. Generation”, 2025), 대표점 64개로 512개짜리보다 나은 결과를 낸 연구도 있다(COD-VAE, 2025). 즉 “논문에서 512를 쓰니 512가 좋다”는 학습 형상이 훨씬 많을 때의 이야기다.

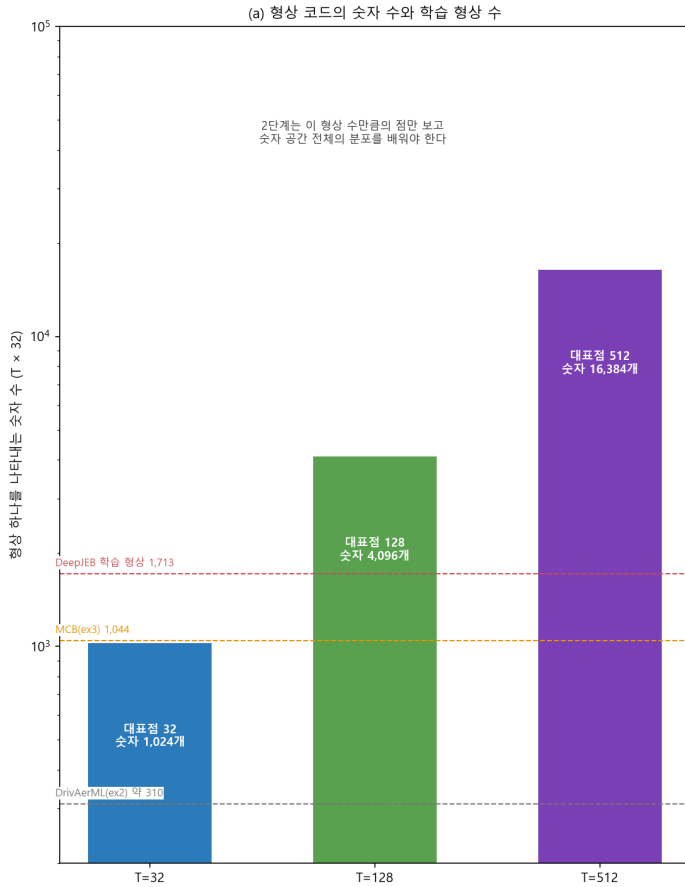
가설과 처방. J1의 1단계는 멀쩡하므로(실제 형상을 그대로 복원하면 J3와 같은 81%가 한 조각) 문제는 2단계다. 2단계에 무엇이 모자란지 하나씩 바꿔 보았다(그림 26의 (b)). K로 시작하는 실험은 J1, J3, J4의 1단계를 그대로 두고 2단계만 새로 학습했다.

- **오래 학습:** 2단계를 500바퀴 대신 2000바퀴(K1, K4).
- **코드 여러 개:** 형상마다 형상 코드를 하나(평균)만 쓰지 않고, 1단계가 내는 평균과 퍼짐 정도에서 8번 뽑아 2단계 자료를 8배로 늘린다(K2, K13, K14). 1단계를 학습할 때 같은 흔들림을 넣어 복원을 배웠으므로 뽑은 코드도 같은 형상으로 복원된다. 지금 config에는 이 기능이 없어 aarl의 임시 스크립트로 했다.
- **시점 이동:** 2단계는 학습할 때 진행 정도 t 를 고르게 뽑는다. 숫자가 많으면 잡음이 많은 초반이 더 어려우므로 t 를 잡음 쪽으로 치우쳐 뽑는다(`fm_time_sampling` `logit_normal`, `fm_time_logit_mean` -1.386 , $T = 128$ 은 -0.693)(K6, K9, K11).
- **큰 2단계:** 대표점마다 다루는 숫자를 256개에서 512개로, 계산 단계를 8개에서 12개로(`fm_hidden` 512, `fm_blocks` 12; K3).
- **위치 표식 제거:** 2단계는 대표점 자리마다 고정된 표식을 더하는데, 대표점에는 순서가 없으므로 이 표식이 방해인지 보았다(K5, K7, K8). 이것도 임시 스크립트로 했다.
- **셋 다:** 코드 8개 + 시점 이동 + 큰 2단계, 250바퀴(K15, 모델 씨앗만 바꾼 K17). 같은 조합을 $T = 128$ 에(K16).

다른 두 데이터에서는 두 단계를 모두 대표점 32와 512로 새로 학습하고(X3, X2), 512의 1단계에 2단계만 시점 이동(X3s, X2s), 코드 8개 + 시점 이동(X3a, X2a)으로 다시 학습했다. 평가는 § 9.6과 같은 프로그램, 같은 형상 쌍, 같은 조건으로 했다.

흔들림. 평가 개수가 24-32개라 형상 하나가 3-4%다. 같은 설정을 다시 학습한 쌍으로 보면, J3와 K4(2000바퀴지만 가장 좋은 지점이 470바퀴라 사실상 J3를 다시 돌린 것)는 9점 이내로 달랐지만, 모델 씨앗만 다른 K15와 K17은 조건 없는 생성이 84%와 66%, 거꾸로 걸기가 71%와 92%로 20점 가까이 달랐다. 그러므로 이 절에서는 20점 안쪽의 차이는 판정하지 않고, K15와 K17은 둘을 함께 본다.

새 지표: 학습 형상 쪽 쏠림. 표 16의 “베끼기 비율”은 숫자가 16384개가 되면 쓸 수 없다. 형상 코드 사이 거리가 모두 비슷해져서, 점검용 형상 가운데 가장 가까운 5%의 거리가 중앙값과 거의 같아지기 때문이다(DrivAerML 512에서 1.336 대 1.358). 그래서 거리의 크기가 아니라 자리를 본다. 생성한 형상마다 가장 가까운 학습 형상까지의 거리를 재어 그 중앙값을 구하고, 학습에 쓰지 않은 점검용 형상에서 같은 거리를 낸 분포와 비교한다. 쏠림이 0이면 생성 형상이 처음 보는 형상만큼 학습 형상과 떨어져 있고, 1이면 점검용 형상 중 학습 형상에 가장 가까운 5%만큼 붙어 있다. 클수록 학습 형상을 따라 한다는 뜻이고,



(b) 돌리는 실험 (kl 1e-8 공통, GPU 3 미사용)

GPU 0	K1	J1(대표점 512)의 1단계 그대로 + 2단계 2000바퀴 → 오래 배수면 따라잡나
GPU 0	K2	J1 + 형상마다 코드 8개 붙여 2단계 500바퀴 → 2단계 자료를 늘리면
GPU 0	K3	J1 + 2단계 모델 커움(폭 512, 중 12) 1000바퀴 → 2단계가 작았나
GPU 0	K4	J3(대표점 32) + 2단계 2000바퀴 → 공정 비교용
GPU 1	J4	DeepIEB 대표점 128, 1단계 500 + 2단계 500 → 중간값
GPU 2	X3	MCB(ex3) 대표점 512, 1단계 1000 + 2단계 500 → 기준: 기존 대표점 32
GPU 0	X2	DrivAerML(ex2) 대표점 32 → 512, 1단계 1500 + 2단계 500 → 세 번째 데이터
GPU 1	K5	J1 + 2단계의 대표점 위치 표시 제거 → 순서 없는 대표점에 위치 표시가 방해인가
GPU 1	K6	J1 + 시점 이동(잡음 많은 쪽을 더 자주 연습) → 큰 코드가 맞춘 학습 시험 배로
GPU 1	K7	J1 + K5와 K6 둘 다 → 두 방법이 겹치면
GPU 1	K8	J3(대표점 32) + 위치 표시 제거 → 32에서도 같은가
GPU 1	K9	J1 + 시점 이동 2배 → 이동을 더 늘리면
GPU 1	K10	J1 + 시점 이동 + 느린 평균(0.999) → 마무리 평균을 길게
GPU 1	K11	J4(대표점 128) + 시점 이동(절반 크기) → 128에서 이동 효과
GPU 2-0	X3s-X2s	X3-X2의 512 1단계 + 시점 이동 → 다른 데이터에서 이동 효과
GPU 2	K12	J1 + 코드 8개 + 시점 이동 → 가장 센 두 처방을 합치면
GPU 2	K13	J1 + 코드 8개, 250바퀴(잡음 수를 K1과 같게) → 절음 수가 같아도 나쁜가
GPU 1	K14	J3(대표점 32) + 코드 8개 → 32도 자료가 모자랐나
GPU 1	K15	J1 + 큰 2단계 + 코드 8개 + 시점 이동, 250바퀴 → 모든 처방을 합치면
GPU 1-0	X3a-X2a	X3-X2의 512 1단계 + 코드 8개 + 시점 이동 → 다른 데이터에서 합친 처방
GPU 1	K16	J4(대표점 128) + K15 처방(이동은 절반 크기) → 128에 같은 처방이면
GPU 0	K17	K15와 같고 모델 씨앗만 7로 → K15가 우연이 아닌가

평가: J1-J3와 같은 방법(한 조각 비율, 조건 오차, 섞기). 1단계 설정은 대표점 수 외에 같다.

그림 26: 대표점 수 연구의 설정. (a) 형상 코드의 숫자 개수($T \times 32$)와 데이터별 학습 형상 수. $T = 512$ 에서는 숫자가 형상 수보다 10~50배 많다. (b) 돌린 실험. K는 이미 학습한 1단계(J1: 512, J3: 32, J4: 128)에 2단계만 새로 학습한 것, X는 다른 데이터에서 학습한 것이다. 벌점 가중치는 모두 $1e-8$ 이고, 1단계 설정은 대표점 수 말고는 같다.

음수는 학습 형상에서 오히려 더 멀다는 뜻이다. 거리의 분포를 데이터·대표점 수마다 따로 쓰므로 대표점 수가 달라도 비교할 수 있다. 다만 중앙값 하나로 보는 것이라, 베꼈다는 증거라기보다 기울기다.

10.1 DeepJEB 결과

처방	조건 없이	실제 조건	부피 오차	쓸림	잡음 쌍 중간	거꾸로 걷기	코드 직선 섞기
J3 32, 기본	72	100	2.5%	-0.10	67	96	88
K4 32, 2000바퀴	81	91	2.6%	0.09	75	96	88
K8 32, 위치 표식 제거	91	100	5.7%	-0.84	79	96	88
K14 32, 코드 8개	81	94	2.9%	0.08	77	92	88
J4 128, 기본	59	72	4.5%	0.12	67	88	96
K11 128, 시점 이동	59	75	4.0%	0.28	71	92	96
K16 128, 셋 다	72	78	1.3%	0.19	67	92	96
J1 512, 기본	31	38	8.3%	0.62	15	54	79
K1 512, 2000바퀴	38	53	5.6%	0.31	38	67	79
K5 512, 위치 표식 제거	12	41	7.5%	0.62	19	42	79
K6 512, 시점 이동	44	69	3.2%	0.66	56	71	79
K9 512, 시점 이동 2배	41	56	3.7%	0.74	35	62	79
K2 512, 코드 8개	47	62	2.2%	0.16	65	83	79
K13 512, 코드 8개, 250바퀴	38	56	4.5%	0.25	40	83	79
K3 512, 큰 2단계	56	72	2.3%	0.43	71	71	79
K12 512, 코드 8개 + 이동	66	72	2.4%	0.35	77	79	79
K15 512, 셋 다	84	81	2.0%	0.16	79	71	79
K17 512, 셋 다(씨앗 7)	66	84	1.6%	0.26	77	92	79

표 17: DeepJEB 대표점 수 연구. 조건 없이, 실제 조건, 잡음 쌍 중간, 거꾸로 걷기(실제 형상 쌍, 빈 조건), 코드 직선 섞기는 한 조각 비율(%)이다. 부피 오차는 실제 조건(강조 배열 1)을 주었을 때 요청한 부피와의 차이 중앙값, 쓸림은 위에서 정의한 값이다(0 근처가 좋다). 실제 형상을 그대로 복원하면 32와 512는 81%, 128은 94%가 한 조각이다. 코드 직선 섞기는 2단계를 거치지 않으므로 2단계 처방과 관계없다. 생략한 K7(위치 표식 제거 + 이동)과 K10(이동 + 느린 평균)은 K6보다 낫지 않았다.

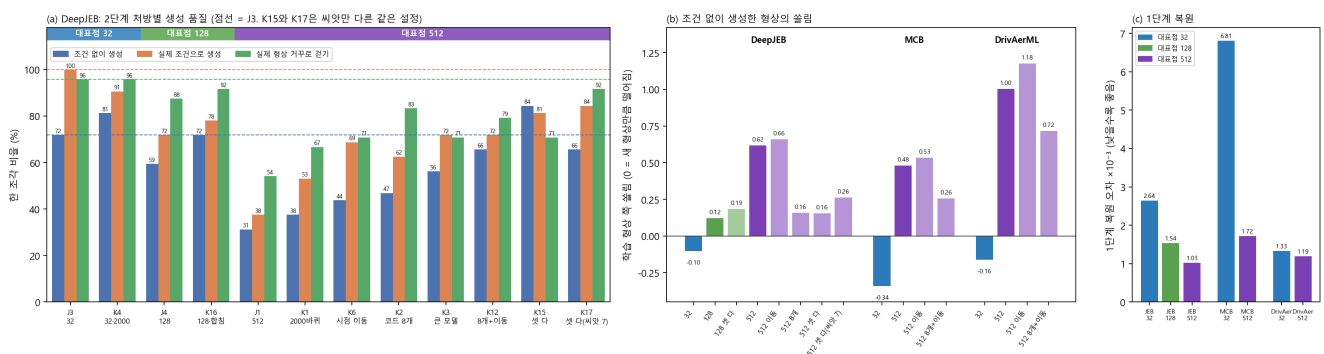


그림 27: 대표점 수 연구의 주요 결과. (a) DeepJEB의 주요 실험. 점선은 J3의 값이다. (b) 조건 없이 생성한 형상이 학습 형상 쪽으로 쓸린 정도 (0 = 처음 보는 형상만큼 떨어짐). 진한 막대는 기본 설정, 옅은 막대는 처방을 더한 것이다. (c) 1단계 마지막 바퀴의 점검용 복원 오차.

512는 2단계 처방으로 살아난다. 표 17와 그림 27의 (a)에서, 처방 하나만 쓴 실험 중 J1(31%)을 가장 많이 올린 것은 큰 2단계(56%), 코드 8개(47%), 시점 이동(44%) 순이다. 오래 학습만 한 K1은 38%로 거의 그대로였고, 1440바퀴 무렵 학습이 튀어 버렸다. 코드 8개는 한 바퀴의 걸음 수가 8배이므로 걸음 수를 K1과 같게 맞추는 K13(250바퀴)도 보았는데, 조건 없는 생성은 38%로 K1과 같았다. 즉 코드 8개의 효과 일부는 걸음 수에서 온다. 그러나 쓸림은 다르다. 시점 이동이나 큰 2단계는 쓸림을 줄이지 못했고 (0.66, 0.43), 코드 8개를 쓴 실험은 쓸림이 0.16-0.35로 J1(0.62)의 절반 안팎이 되었다(그림 27의 (b)).

형상 1713개의 코드 하나씩만 보면 2단계가 그 점들을 외우고, 형상마다 흔들린 코드 여러 개를 보면 점 사이를 채우는 법을 배운다고 볼 수 있다. 셋을 합친 K15·K17은 조건 없는 생성이 84%·66%, 잡음 쌍 섞기가 79%·77%로 J3·K4(72%·81%, 67%·75%)와 같은 수준이고, 부피 오차는 2.0%·1.6%로 J3·K4(2.5%·2.6%)보다 조금 작다. 하지만 실제 조건으로 생성한 한 조각 비율은 81%·84%로 J3·K4(100%·91%)보다 낮고, 거꾸로 걷기는 71%·92%로 흔들림이 커서 판정하지 않는다.

32에는 같은 처방이 거의 듣지 않는다. J3에 코드 8개(K14)나 2000바퀴(K4)를 주어도 모두 흔들림 안이다. 2000바퀴 학습은 470바퀴 무렵이 가장 좋았고 그 뒤로는 점검용 오차가 다시 올라, 32의 2단계는 오히려 오래 학습하면 학습 형상에 맞춰진다. 위치 표식을 뺀 K8(91%)은 한 조각 비율이 가장 높지만 부피 오차가 5.7%로 두 배가 되고 쓸림이 -0.84 (처음 보는 형상보다도 학습 형상에서 멀다)여서, 학습한 형상들의 범위 밖으로 벗어난 형상을 만드는 것으로 보인다. 개선이라 하기 어렵다. 반대로 512에서 위치 표식을 빼면(K5) 12%로 크게 나빠졌다. 대표점을 고르는 순서(멀리 떨어진 점을 차례로 고름)가 거친 것에서 세밀한 것으로 가는 순서여서, 자리 표식이 이 정보를 전하는 것으로 보인다.

128은 복원 경로가 가장 좋고, 처방을 더하면 균형이 가장 좋다. J4(128)는 실제 형상을 그대로 복원했을 때 94%, 코드 직선 섞기 중간에서 96%가 한 조각으로 32와 512(81%, 79–88%)보다 높다. 복원 오차도 1.54×10^{-3} 로 32(2.64×10^{-3})의 1.7분의 1이다. 기본 2단계로 새로 생성하면 59%로 32보다 낮지만, 셋을 합친 K16은 조건 없는 생성 72%(J3와 같음), 거꾸로 걷기 92%, 부피 오차 1.3%(모든 실험 중 가장 작음), 쓸림 0.19다. 남은 약점은 실제 조건으로 생성한 한 조각 비율(78% 대 100%)이고, 씨앗 하나로만 확인했다.

10.2 다른 데이터에서 확인

같은 비교를 MCB(ex3, 너트 1044개)와 DrivAerML(ex2, 자동차 약 310개)에서 두 단계 모두 새로 학습해 확인했다. 두 데이터 모두 대표점 수 말고는 기존 설정과 같고, 벌점 가중치는 $1e-8$ 이다. MCB의 대표점 32는 벌점 실험의 kl $1e-8$ 모델(§ 3.7)이다.

데이터	모델	복원 오차	조건 없이	조각 수	실제 조건	부피 오차	쓸림	거꾸로 걷기
MCB	32	6.81×10^{-3}	81	1.4	78	7.7%	-0.34	74
MCB	512 (X3)	1.72×10^{-3}	19	6.7	53	8.9%	0.48	54
MCB	512 + 이동 (X3s)	같음	56	5.0	59	4.1%	0.53	42
MCB	512 + 8개 + 이동 (X3a)	같음	72	1.9	66	2.0%	0.26	75
DrivAerML	32	1.33×10^{-3}	–	10.3	–	–	-0.16	–
DrivAerML	512 (X2)	1.19×10^{-3}	–	12.2	–	–	1.00	–
DrivAerML	512 + 이동 (X2s)	같음	–	11.3	–	–	1.18	–
DrivAerML	512 + 8개 + 이동 (X2a)	같음	–	11.9	–	–	0.72	–

표 18: 다른 데이터의 대표점 32 대 512. 복원 오차는 1단계 마지막 바퀴의 점검용 오차, 조각 수는 조건 없이 생성한 형상 하나의 평균 조각 수다. 한 조각 비율과 부피 오차의 뜻은 표 17와 같다. DrivAerML은 여러 부품으로 된 자동차라 한 조각 비율이 뜻이 없고, 조건이 형상 치수가 아니라 부피 오차도 없다. 거꾸로 걷기는 실제 형상 쌍, 빈 조건이다.

MCB. 복원은 512가 약 4배 좋다(1.72×10^{-3} 대 6.81×10^{-3}). 그러나 같은 2단계로 생성하면 DeepJEB보다 더 크게 나빠진다. 조건 없이 만든 형상의 19%만 한 조각이고 형상 하나에 평균 6.7조각이 붙는다(32는 81%, 1.4조각). 시점 이동만 더하면 56%로 오르지만 쓸림은 그대로이고(0.53), 코드 8개까지 더한 X3a는 72%, 1.9조각, 쓸림 0.26으로 DeepJEB의 K12·K15와 같은 모습이다. X3a는 실제 조건의 부피 오차가 2.0%로 32(7.7%)보다 훨씬 작지만, 실제 조건으로 생성한 한 조각 비율은 66%로 32(78%)보다 낮다. 조건까지 함께 옮기는 거꾸로 걷기도 42%로 32(75%)보다 낮다. 즉 MCB에서도 처방이 512를 32 가까이 끌어올리고 조건은 더 정확히 따르게 하지만, 형상이 한 덩어리로 나오는 비율은 32를 넘지 못한다.

DrivAerML. 자동차는 차체, 바퀴, 거울처럼 여러 부품이어서 실제 형상을 그대로 복원해도 6%만 한 조각이다. 생성한 형상의 조각 수(10-12개)와 부스러기 비율(약 2%)도 모든 모델에서 비슷해, 조각으로는 모델을 가를 수 없다. 복원도 512가 1.1배 좋을 뿐이다. 남는 차이는 쓸림이다. 32는 -0.16 으로 처음 보는 형상만큼 떨어져 있지만, 512는 1.00(점검용 형상 중 가장 가까운 5%만큼 학습 형상에 붙음)이고, 32개 중 3개(9%)는 어느 점검용 형상보다도 학습 형상에 가깝다(32는 0개). 시점 이동은 이를 줄이지 못했고(1.18, 3개), 코드 8개 + 시점 이동도 0.72(2개)에 그쳤다. 학습 형상이 310개뿐이면 512의 2단계는 처방을 해도 학습 형상 쪽으로 쏠린다.

비용. aarl에서 다른 작업과 GPU를 나눠 쓴 시간이라 대략값이다. 1단계는 512가 32의 약 2-2.4배 걸렸다(MCB 1000바퀴: 약 11시간 대 27시간). 2단계는 J3 500바퀴가 약 13분인데, 셋을 합친 K15(250바퀴)는 약 4시간으로 약 20배, 128의 K16은 약 1시간 20분이었다.

10.3 결론: 대표점 수

1. 복원만 보면 32는 작다. 512의 점검용 복원 오차는 DeepJEB에서 2.6분의 1, MCB에서 4분의 1이다(DrivAerML은 1.1분의 1). “대표점이 많을수록 좋다”는 이 점에서는 맞다.
2. 생성까지 보면 지금 데이터 크기에서 32는 작지 않다. 2단계를 같게 두면 512는 세 데이터 모두에서 나뉘었다(한 조각 비율 DeepJEB 31% 대 72%, MCB 19% 대 81%, DrivAerML 쏠림 1.00 대 -0.16). 문헌의 512 이상은 학습 형상이 이보다 100-1000배 많을 때의 결과다.
3. 512는 2단계 처방을 하면 쓸 만해진다. 그러나 32보다 낫지는 않다. 코드 8개 + 시점 이동 + 큰 2단계를 합치면 DeepJEB·MCB에서 조건 없는 생성이 32와 같은 수준으로 오르고 부피 오차는 오히려 작아진다(DeepJEB 1.6-2.0% 대 2.5%, MCB 2.0% 대 7.7%). 하지만 실제 조건으로 생성한 한 조각 비율은 여전히 낮고(DeepJEB 81-84% 대 91-100%, MCB 66% 대 78%), 2단계 시간이 약 20배, 1단계 시간이 약 2배다. 가장 중요한 처방은 코드 8개다. 이것만이 학습 형상 쪽 쏠림을 줄였다. 형상이 310개뿐인 DrivAerML에서는 처방을 해도 쏠림이 남았다.
4. 128 + 처방(K16)이 가장 균형이 좋다. 복원 1.7배, 실제 형상 복원·코드 직선 섞기 94-96%, 조건 없는 생성은 J3와 같은 72%, 부피 오차 1.3%로 가장 작다. 남은 약점은 실제 조건으로 생성한 한 조각 비율(78%)이고, 씨앗 하나로만 확인했다.
5. 더 좋게 하려면. (가) 기본은 32를 유지한다. (나) 치수를 정확히 맞추는 것이 중요하면 128에 시점 이동(`fm_time_logit_mean -0.693`)과 큰 2단계(`fm_hidden 512, fm_blocks 12`)를 쓰고, 형상마다 코드 여러 개를 쓰는 기능을 2단계 학습에 넣는다(지금 config에는 없다. 넣을지는 사용자가 정한다). (다) 512는 학습 형상이 수만 개 이상일 때 다시 본다. (라) 평가 개수 32개와 씨앗 하나로 20점 차이도 흔들림일 수 있으므로, 설정을 정할 때는 평가 형상 128개 이상, 씨앗 2-3개로 다시 켜다.

11 권장 사항 (아직 적용하지 않음)

아래는 모두 제안이다. config, 코드, 데이터는 하나도 바꾸지 않았다.

1. ex3는 방향 정렬이 먼저다. 전처리에서 모든 너트의 구멍 축을 z로 돌린다(비스듬한 105개는 눈으로 확인). 방향 무리가 사라지고 상자 크기 조건이 순수한 크기 정보가 된다. 데이터를 다시 만드는 일이라 결정은 사용자에게 있다. 무작위 회전을 섞어 학습하는 방법은 방향 무리를 없애지 못하고 연속 회전까지 배우게 만들므로, 보간이 목적이면 정렬이 맞다.
2. DeepJEB config의 대표점 512 / kl 1e-6은 J0에서 붕괴가 확인된 조합이다. 작업본 `configs/SDFFlow/geometry_generation/ex1/baseline/config_train_sdf_flow.txt`와 `paper/deepjeb_design/deepjeb_design.tex`에 이 조합이 있다. 대표점 32 / kl 1e-8(J3)로 바꾸기를 권한다. 대표점 512를 유지하려면 kl을 1e-8 이하로 낮춰야 하고(J1), 그래도 같은 2단계 학습량에서는 생성이 J3보다 나뉘고, 2단계 처방을 합쳐야 J3 수준이 된다(§ 10). 치수를 정확히 맞추는 것이 중요하면 대표점 128 + 2단계 처방(K16)도 후보다. 둘 다 고치지 않았다.

3. **ex3 config**는 지금 **kl 1e-10**이다. 작업본 파일이 1e-10으로 바뀌어 있다(주석 “was 1e-4”). 이 문서를 쓰면서 바꾼 것이 아니다. ex3 결과로 보면 1e-10은 조건 없는 생성이 가장 좋아서 합리적인 선택이다.
4. 2단계는 약 700바퀴 + **fm_best.pth**. 1단계는 700-900바퀴면 충분하다.
5. 조건 강조 배율은 1을 기본으로. 치수를 꼭 맞춰야 할 때만 2.
6. **보간 용도별 선택**. 실제 두 형상 사이를 섞을 때에는 “거꾸로 걷기 + 조건도 옮김”이 가장 고르다. 한 속성(부피 등)을 따라 움직이는 탐색에는 **cond_sweep**이 맞지만 두 끝의 방향이 같을 때만 매끄럽다. 조건 하나로 고정한 잡음 섞기는 형상이 거의 안 변해 탐색에 쓸모가 적다. 실제 형상 사이 섞기를 프로그램 기능으로 넣을지는 사용자 결정이다.
7. **수가 적은 종류를 더 자주 보기**. 캐슬 181, 홈 59, 사각 45개는 적다. 종류별로 고르게 뽑아 학습하면 이 종류들의 오차가 줄 수 있다. 형상 크기를 조금씩 바꾼 사본을 섞어 학습하는 방법, 대표점 64-128 개도 후보다(§ 10).
8. 복원 용도라면 **latent_refine_steps**를 켜다. 단, 고친 코드는 2단계에 넣지 않는다.
9. 떨어진 조각은 **body_count_raw**로 확인한다. 가장 큰 조각만 남기는 동작을 config로 끌 수 있게 할지는 사용자 결정이다.
10. **종류 조건은 뒤로 미룬다**. 두 끝 방향이 같은 쌍끼리 보면 다른 종류 쌍(CROSS)이 같은 종류 쌍(SAME)보다 조금 더 튀지만(팀 중앙값 3.14 대 2.21), 방향이 같고 다름에 따른 차이(2.5 대 5.3)가 훨씬 크므로(표 10) 지금 팀의 주원인은 종류가 아니라 방향이다. 방향을 정렬한 뒤에도 잠금 조건에서 육각이 나오는 일이 남으면 그때 **condition_names**에 종류를 넣는다.
11. **DeepJEB 점 밀도**. J1-J3의 점검용 오차는 학습용과 크게 벌어지지 않았다(J3 1.4배, J1·J2는 학습용이 조금 더 큼). 지금은 형상당 점을 다시 만들 필요가 없다.
12. **대표점을 128 이상으로 쓰면 2단계 처방을 함께 쓴다**. 학습 시점 치우침(**fm_time_sampling** **logit_normal**, **fm_time_logit_mean** -0.693)과 큰 2단계(**fm_hidden** 512, **fm_blocks** 12)는 config만으로 된다. 가장 효과가 큰 “형상마다 코드 여러 개”는 지금 코드에 없어 2단계 학습을 고쳐야 하므로 결정은 사용자에게 있다(§ 10.3).

12 한계

- 경로 수가 묶음마다 8-16개로 적다. 정상 모델 세 개(1e-6, 1e-8, 1e-10) 사이의 차이는 우연 범위 안일 수 있다.
- 모든 실험이 난수 고정값 하나, 데이터 나누기 하나로 돌아왔다.
- 128³ 격자는 얇은 벽을 다 담지 못한다. 캐슬은 정답도 한 조각 비율이 66.7%다.
- ex3 점검용에는 학습용과 같은 형상이 섞여 있어(27/130) 점검용 수치가 후하게 나왔다.
- 구멍 수 오르내림의 원인은 아직 확정하지 못했다. DeepJEB 결과로 판단한다.
- 모든 판정은 기하 모양 기준이다. 구조 성능(해석 결과)은 보지 않았다.
- DeepJEB 결과는 아직 없다(§ 9).

A 풀어 쓴 말과 흔히 쓰는 용어

이 문서의 말	흔히 쓰는 용어	관련 config key
형상 압축 모델 (1단계) 요약부 / 복원부 형상 코드 대표점	VAE (variational autoencoder) encoder / decoder latent (잠재 변수) token, query token	<code>vae_*</code> <code>encoder_*</code> , <code>decoder_*</code> <code>latent_tokens</code> , <code>latent_dim</code> <code>latent_tokens</code> , <code>encoder_query_type</code>
대표점마다 숫자 32개 훑어보기, 정보 나누기	channel, latent dimension cross-attention, self-attention	<code>latent_dim</code> <code>encoder_heads</code> , <code>encoder_self_attention</code>
사인 · 코사인 늘리기 평균 / 퍼짐 정도 / 흔들림	Fourier feature μ / log-variance / reparameterization noise	<code>fourier_bands</code> <code>posterior_noise_*</code> , <code>posterior_min_std_rel</code>
퍼짐 벌점 코드 붕괴 살아 있는 칸 생성 모델 (2단계) 50걸음 이동 이동 방향 빈 조건 조건 강조 배율 가중치 평균본	KL divergence posterior collapse active units (ActiveSNR) flow matching, rectified flow, DiT ODE 적분 (Euler) velocity field null condition, condition dropout classifier-free guidance (CFG) EMA (exponential moving average)	<code>kl_weight</code> , <code>kl_warmup_epochs</code> — 학습 기록 <code>fm_*</code> <code>ode_steps</code> — <code>cond_dropout</code> , <code>cond_dropout_mode</code> <code>cfg_scale</code> <code>vae_use_ema</code> , <code>fm_use_ema</code>
바퀴	epoch	<code>vae_training_epochs</code> , <code>fm_training_epochs</code>
한 번에 보는 형상 수 학습 속도 계산 정밀도 낮추기 학습용 / 점검용 / 시험용 부호 거리 값 0인 면 추출 조각 / 떨어진 조각	batch size learning rate AMP (mixed precision) train / validation / test SDF (signed distance function) Marching Cubes connected component, body / stray	<code>vae_batch_size</code> , <code>fm_batch_size</code> <code>vae_learningr</code> , <code>fm_learningr</code> <code>vae_use_amp</code> , <code>fm_use_amp</code> <code>eval_split</code> <code>clamp_dist</code> <code>mc_resolution</code> <code>body_count_raw</code>
구멍 수 겹침률 표면 평균 거리 원호 섞기 / 직선 섞기 거꾸로 걸어 잡음 찾기 조건만 옮기기 순위 상관 점 분포의 주축 서로 가장 멀리 떨어진 점 고르기 코드 직접 고치기	genus IoU (intersection over union) Chamfer distance slerp / lerp ODE inversion condition sweep Spearman 상관계수 PCA 주성분 farthest point sampling (FPS) latent optimization	— — — <code>interpolation_space</code> — <code>cond_sweep</code> — — <code>encoder_query_type fps</code> <code>latent_refine_*</code>

B ex3 학습 config 원문

`configs/SDFFlow/geometry_generation/ex3/baseline/config_train_sdfflow.txt`(현재 작업본)의 key 부분이다. 머리 주석은 생략했다. DeepJEB(**ex1**) 파일과 다른 줄은 오른쪽에 적었다.

```

model          sdfflow
mode           train
gpu_ids        1                % ex1: 7
parallel_mode  single
seed          42

```

split_seed	42	
dataset_dir	../../dataset/geometry_generation/ex3_mcb.h5	
	% ex1: ../../dataset/geometry_generation/ex1_deepjeb.h5	
split_by_parent	False	% ex1: True
num_encoder_points	6144	
num_query_points	8192	
latent_tokens	32	% ex1: 512
latent_dim	32	
encoder_query_type	fps	
encoder_dim	256	
encoder_heads	8	
encoder_blocks	4	
encoder_self_attention	True	
decoder_type	attention	
decoder_hidden	256	
decoder_layers	4	
decoder_heads	8	
fourier_bands	8	
kl_weight	0.0000000001	% ex1: 0.000001
clamp_dist	0.1	
deterministic_warmup_epochs	50	
posterior_noise_warmup_epochs	100	
posterior_noise_max_scale	1	
kl_warmup_epochs	200	
posterior_min_std_rel	0.05	
surface_weight	1	
normal_weight	0.1	
eikonal_weight	0.1	
hybrid_grad_points	1024	
use_conditions	True	
condition_names	bbox_x, bbox_y, bbox_z, volume, area	% ex1: volume, area
min_condition_std	0.00001	
condition_clip	5	
cond_dropout	0.2	
cond_dropout_mode	all	
fm_arch	dit	
fm_hidden	256	
fm_blocks	8	
fm_heads	8	
fm_cond_hidden	128	
fm_time_sampling	uniform	
encode_batch_size	8	
ode_steps	50	
vae_training_epochs	1000	% ex1: 1500
vae_batch_size	8	
vae_learningr	0.0001	
vae_weight_decay	0.0001	
vae_warmup_epochs	20	
vae_num_workers	2	
vae_use_amp	False	
vae_use_ema	True	
vae_ema_decay	0.99	
vae_val_interval	5	
vae_test_interval	100	
vae_num_test_shapes	2	
vae_mc_resolution_test	64	
fm_training_epochs	500	
fm_batch_size	64	
fm_learningr	0.0001	
fm_weight_decay	0.0001	
fm_warmup_epochs	10	
fm_num_workers	0	
fm_use_amp	True	

```

fm_use_ema                True
fm_ema_decay              0.99
fm_val_interval           5
fm_test_interval          50
fm_num_test_shapes        2
fm_mc_resolution_test     64
display_testset           True
skip_completed_stages     True
output_dir                ../../output/geometry_generation/ex3_mcb/sdfflow/samples
vae_modelpath             ../../output/geometry_generation/ex3_mcb/sdfflow/vae.pth
fm_modelpath              ../../output/geometry_generation/ex3_mcb/sdfflow/fm.pth
fm_best_modelpath         ../../output/geometry_generation/ex3_mcb/sdfflow/fm_best.pth
pipeline_log_file         ../../output/geometry_generation/ex3_mcb/sdfflow/pipeline.log
vae_log_file_dir          ../../output/geometry_generation/ex3_mcb/sdfflow/vae.log
fm_log_file_dir           ../../output/geometry_generation/ex3_mcb/sdfflow/fm.log
% ex1 경로: ../../output/dataset_matrix/geometry_generation/ex1/sdfflow/...

```